Karavaan

Kaavish Report
presented to the academic faculty
by

Rasib Qazi	rq08098
Ansab Chaudhary	mc08077
Hussain Umer	hf08040
Hussain Mustansir	hm08436

In partial fulfillment of the requirements for
Bachelor of Science
Computer Science

Dhanani School of Science and Engineering

Habib University

Spring 2026

Karavaan

This Kaavish project was supervised by:

My Internal Supervisor
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____.

Dedication

We dedicate this work to our families, whose constant support, encouragement, and belief in us made this journey possible. Their patience and motivation sustained us through the challenges of this project.

We also dedicate this project to our mentors and faculty at Habib University for their guidance and academic support, which played a crucial role in shaping our learning and execution.

Finally, we extend this dedication to our industry partner, Spursol, for providing us with the opportunity to work on a real-world problem. Their collaboration, feedback, and practical insights helped bridge the gap between academic learning and industry application, making this project a meaningful and impactful experience.

Acknowledgements

We would like to express our sincere gratitude to our academic supervisor, Qasim Pasta, for his continuous guidance, support, and valuable feedback throughout the course of this project. His mentorship played a crucial role in shaping both the direction and quality of our work.

We are equally thankful to our industry supervisor, Kiran, for providing us with the opportunity to work on a real-world problem and for her consistent support, insights, and coordination throughout the project.

We also extend our appreciation to Anas, our technical lead from the industry, for his hands-on guidance, technical expertise, and assistance in overcoming practical challenges during development.

Finally, we would like to thank the faculty of the Computer Science department at Habib University for their support and for providing an environment that encouraged learning and innovation.

Abstract

Karavaan is a mobile platform designed to support community-driven initiatives by bringing volunteers, donors, and organizations together in a single digital space. Many community activities today are fragmented across different platforms such as WhatsApp groups, social media posts, and informal networks, which makes it difficult for people to discover events, support fundraisers, or coordinate efforts effectively.

The proposed system solves this problem by providing a centralized application where users can create and join communities, organize events, raise funds, and communicate in real time. The platform includes features such as community discussions, event management with RSVP functionality, fundraising with transparent donation tracking, and real-time messaging for collaboration.

The system is implemented using a scalable architecture that integrates a mobile user interface with backend services, a PostgreSQL database, and external services such as payment gateways, notifications, and location APIs. By consolidating these capabilities into a single platform, Karavaan aims to simplify community coordination, improve transparency in donations, and increase participation in social causes.

The project demonstrates how technology can strengthen civic engagement by making it easier for people to discover opportunities, collaborate with others, and contribute to meaningful social impact.

Contents

List of Figures

List of Tables

1. Introduction

2. Introduction

2.1 Background

Civil society organizations, volunteers, and donors in Pakistan rely on a fragmented set of platforms to coordinate community work. Announcements are shared over WhatsApp groups, fundraisers are hosted on unverified pages, and event coordination happens across disconnected tools. This fragmentation creates friction at every step: volunteers cannot discover opportunities easily, donors have no reliable way to verify causes, and NGOs struggle to mobilize people consistently.

2.2 Problem Statement

Social groups and volunteers use many separate platforms to organize causes, events, and donations. This makes it hard to find trusted fundraisers, coordinate people, and see how donations are used, which reduces participation and impact.

2.3 Proposed Solution

Kaarvan is a mobile platform that consolidates community organizing, volunteer coordination, fundraising, and donor transparency into a single application. Built with React Native and Expo, it provides communities, events, discussions, real-time chat, and AI-powered recommendations to connect the right people with the right causes.

2.4 Scope

The current scope of the project covers community creation and management, event organization and RSVP, fundraiser listings, a real-time discussion and messaging system, user authentication via Google Sign-In, and personalized recommendations. Deployment and formal user testing are planned as part of the next phase in collaboration with Spursol.

3. Literature Review

Literature Review – Kaavish

Introduction:
Karavaan is a centralized platform that connects people with verified social causes. Today, most community service efforts are spread across WhatsApp groups, Instagram posts, and offline networks, making it hard for people to discover, trust, and support meaningful initiatives. Our mobile app aims to bring these efforts into one reliable space. In this literature review, we examine existing platforms working in the same space and explore how similar features have been designed and implemented in past research and applications.

Competitor Analysis

Global Fundraising and Donation Platforms

There are a number of global fundraising platforms already operating at a large scale. GoFundMe has established itself as the dominant "giving layer of the internet", achieving a near-monopoly by raising over $17 billion across 200 million donations by mid-2022. This immense scale translates to approximately 90% of the social crowdfunding market in the United States and 80% globally. This dominance was secured through the strategic acquisition of major competitors (e.g., YouCaring and CrowdRise) and leveraging a "frictionless" user experience that allows users to donate with few clicks [1]. Conversely, platforms like GlobalGiving focus on grassroots projects, having facilitated over $1 billion since 2002 [2].

Volunteer Management Platforms

VolunteerMatch is the web's largest volunteer engagement network, helping over 15 million people annually find opportunities at 113,000+ US organizations since 1998 [3]. Golden provides AI-enabled volunteer-to-donor conversion tools with advanced CRM integrations [4]. GivePulse offers a single platform for organizations to list and manage volunteer events while communicating with members and donors [5]. POINT provides a free, all-in-one volunteer management platform and app with website integration [6].

Pakistan-Specific Platforms

Transparent Hands leads Pakistan's online donation landscape with crowdfunding for patient surgeries, offering complete transparency through patient selection and regular progress updates [7]. Easypaisa Donations enables giving to 50+ organizations including Edhi, Alkhidmat, and Shaukat Khanum through their mobile banking app [8]. LaunchGood serves as the world's largest Muslim crowdfunding platform with 0% platform fees [9]. Alkhidmat Foundation and JDC also offers integrated online donations for various causes through multiple payment channels.

Current State of the Art Technologies That We Can Use to Implement

Recommendation and Personalization Algorithms

Modern social impact platforms employ sophisticated recommendation systems to connect users with relevant causes, events, and fundraisers. Collaborative filtering remains the foundational approach, analyzing user-item interactions to predict preferences based on similar users' behaviors [10].

Beyond foundational methods, advanced collaborative filtering can be tailored for social impact by integrating social trust. Li and Lin (2025) propose a hybrid model (STUBCF) that incorporates implicit social and trust relationships to calculate user similarity before applying spectral clustering. This method mines user preferences within these socially-defined clusters, resulting in highly localized recommendations with reduced prediction error [11].

This evolution is advancing towards sophisticated, multi-faceted models. For instance, the Community-Aware Social Community Recommendation (CASO) model explicitly encodes global social patterns, local closeness, and collaborative preferences

into a unified framework, outperforming traditional methods [12]. The trend is a shift from simple behavior-matching to socially-aware, context-rich personalization, making these systems uniquely powerful for activism and donation ecosystems where shared purpose is paramount.

For a platform like Karavaan, these approaches are critical, as engagement with causes is often driven by social influence and trust within a community, not just past donation history.

Location-Based Event Recommendation Systems

Al-Ghobari et al. (2021) proposed the Location-Aware Personalized Traveler Assistance (LAPTA) system, which integrates user preferences with GPS technology to generate personalized recommendations. Their system uses a K-Nearest Neighbor (KNN) algorithm with collaborative filtering to match location categories with user input. The KNN algorithm was selected because it overcomes scalability problems, is easy to implement using Euclidean distance calculations, and produces accurate recommendations with lower error rates [13]. The system separates data from Google locations into name and category tags, then fetches keywords from user input based on past search behavior.

The LAPTA system achieved a Mean Absolute Error (MAE) of 0.7651 and Root Mean Square Error (RMSE) of 0.8927 with a neighborhood size of 10, and MAE of 0.7184 with RMSE of 0.9309 when using neighborhood size 20 [13]. The adjusted cosine similarity with neighborhood size 20 provided the minimal error values. When compared with related literature, LAPTA outperformed several existing recommendation systems, though one cluster-based approach achieved better results with MAE of 0.2510.

Vedashree and Sandhya (2023) presented an alternative approach that uses fuzzy logic and neural network techniques to model user preferences and provide tailored recommendations for travelers. This system considers multiple factors including the user's location, preferences, time of day, weather conditions, and local events to create personalized suggestions. The recommendation process involves user profiling that maintains records of each user's preferences, interests, and previous behaviors, combined with geolocation tracking through GPS, Wi-Fi, or cellular networks [14]. The system aims to deliver highly personalized suggestions that are specific to the user's current or intended location and can provide real-time recommendations by leveraging contextual information such as weather or time of day [14].

Trending Metrics

To effectively identify trending content, platforms like LinkedIn employ a two-stage classification pipeline that analyzes both content and engagement velocity. A proactive deep learning model first assesses static post and author features at submission to predict virality. If content remains active, a reactive model then monitors the temporal sequence and velocity of interactions (likes, shares, comments), using these dynamic engagement features as the primary signal for detecting viral cascades within the network [15].

The TrendNet algorithm addresses the limitations of global trend systems by generating personalized hashtag recommendations based on a user's immediate social network rather than general geographic data [16]. The model constructs a personalized corpus from a user and their followees, then calculates an "Interest Rate" score for each hashtag by combining its frequency within the network with its aggregate engagement (likes and retweets). A key feature is its built-in spam mitigation, which caps the number of hashtags considered per user to prevent artificial inflation of trends and ensure recommendations reflect genuine community interest.

To address the unique dynamics of event-based recommendations, our proposed algorithm moves beyond traditional post engagement metrics by incorporating signals of intent, urgency, and local relevance. The model calculates a trending score based on three core components: (1) a weighted Engagement Score that prioritizes high-intent actions like RSVPs over passive views; (2) the Velocity and Acceleration of this engagement within sliding time windows to detect exponential growth and momentum; and (3) event-specific boost factors for Location Relevance and Time-Sensitivity. This hybrid approach ensures that trending events reflect not just popularity, but also user commitment and immediate, local relevance which ensures a robust implementation of one of Karvaan's core features.

Real Time Messaging and Chat Implementation

Modern real-time chat systems are predominantly built on the WebSocket protocol, which facilitates full-duplex, bi-directional communication over a single TCP connection with minimal latency [17]. Key advantages of WebSocket include its protocol flexibility—supporting JSON, binary, and custom formats—along with technology-agnostic backend implementation and lightweight efficiency that eliminates repetitive HTTP overhead.

While the Extensible Messaging and Presence Protocol (XMPP) offers a robust,

decentralized alternative with built-in presence management and can operate over a WebSocket transport layer, WebSocket maintains a distinct advantage in speed, performance, and data integrity under high concurrency [18].

Consequently, considering the application's functional and non-functional requirements alongside our existing tech stack, WebSocket is the most appropriate choice for implementing the messaging features.

Difference in Karavaan and Existing Work

Karavaan addresses critical gaps in the social impact ecosystem by offering an integrated platform that unifies traditionally isolated functions. Unlike existing platforms that are either donation-only or volunteer-only, Karavaan provides a unified experience for donors, volunteers, and organizers. It solves the problem of fragmented discovery across platforms like WhatsApp and Instagram with a centralized home feed for events, fundraisers, and communities. Donation tracking is also an essential feature that streamlines the processes of charitable organizations. Furthermore, it introduces a verification system to build trust in smaller organizations, a personal "Impact Journal" for tracking contributions, and built-in community spaces for engagement—key differentiators that create a cohesive and transparent social impact environment.

4. Software Requirement Specification (SRS)

4.1 Introduction

4.1.1 Purpose

Kaaravan is a mobile application designed to build a platform for activists, changemakers, artists, and volunteers in Pakistan. It enables users to discover, join, and organize community-driven initiatives and causes such as environmental work, education, equality, or disaster relief. The system focuses on connecting individuals and organizations to collaborate, raise funds, and track social impact efficiently and transparently.

4.1.2 Scope

The app provides the following capabilities:

- Community creation and management.
- Event discovery, participation, and organization.
- Fundraising and donation management with local payment integrations.
- Private and group messaging for collaboration.
- Emergency coordination and cross-community support.

4.1.3 Intended Users

- Volunteers, activists, and general citizens.

- NGOs and community organizations.
- Educational institutions, local authorities, and corporations

4.2 Functional Requirements

4.2.1 User Authentication and Profile Management

- The system shall allow users to sign up by entering full name, an email address, password or through Google sign up.
- The system shall allow users to sign in using their registered credentials or through Google.
- The system shall allow users to reset their password via email OTP verification.
- The system shall allow users to create and edit a personal profile including name, location, and interests.
- The system shall store and display a user's cumulative impact metrics (e.g. total donations made, causes supported).
- The system shall ask CNIC for verification purposes.

4.2.2 Community Features

- The system shall allow users to create a new community by specifying a name, description, profile image, and category (e.g., Environment, Education) and provide an option to invite users on creation.
- The system shall allow users to join or leave communities.
- The system shall categorize members as Community Admin and Member, each with distinct permissions.
- The system shall allow community admin to assign and modify member roles.
- The system shall allow moderator to perform all admin functions except removing the admin
- The system shall allow admin to post announcements, discussions, pin posts, create fundraisers, and create events.

- The system shall allow members to create posts, comment, and like on all posts.
- The system shall allow members to view upcoming events, respond to those events, view fundraisers, and donate to those fundraisers.

4.2.3 Event Management

- The system shall allow community admin to create, edit, and delete events with details (title, description, location, time, capacity).
- The system shall allow users to RSVP to an event and see the number of attendees.
- The system shall send push/email notifications to users when an event they RSVPed for is in the next 24 hours, updated, or canceled.
- The system shall display event details with a map showing the location.
- The system shall allow co-hosting of events between multiple communities.

4.2.4 Fundraising and Donations

- The system shall allow communities to create fundraisers specifying a goal, category, deadline, and description.
- The system shall allow users to donate securely using local payment gateways (credit/debit cards using Switch).
- The system shall display progress toward fundraising goals (e.g., Rs 347,500 raised of Rs 500,000).
- The system shall log and display recent donations and amounts.

4.2.5 Messaging and Communication

- The system shall allow one-on-one messaging between users.
- The system shall allow community group chats for discussions and event coordination.
- The system shall allow moderators to delete inappropriate messages.

4.2.6 SOS and Smart Resource Pooling (SRP)

- The system shall allow users or communities to create SOS alerts specifying the type of emergency (e.g., flood, fire, earthquake).
- The system shall automatically notify relevant users and organizations based on expertise (e.g., doctors, logistics teams).

4.3 Non-Functional Requirements

4.3.1 Security

- All sensitive data (passwords, transactions) shall be encrypted using protocols like AES-256.
- The system shall enforce HTTPS for all network communication.
- The app shall provide two-factor authentication (2FA) for all users.
- Only verified users (CNIC verified) shall be allowed to create communities or manage member roles.

4.3.2 Reliability

- The system shall provide 99.0% uptime availability.
- Data backups shall occur every 72 hours.
- In the event of a crash or outage, the system shall recover within 5 minutes using the latest backup snapshot.

4.3.3 Scalability

- The system shall support up to 10,000 users concurrently.

4.3.4 Usability

- 95% of users shall be able to complete core tasks (sign-up, join community, RSVP event) within 5 clicks.

4.3.5 Data Integrity and Backup

- All user data changes (donations, RSVPs, discussions) shall be committed atomically to ensure consistency.

4.3.6 Performance

- Page load time shall not exceed 3 seconds on standard 4G connectivity.
- The system shall process donations and fund creation requests within 10 seconds.
- Chat messages shall have a delivery latency of less than 300 ms.
- Push notifications shall be delivered to users within 5 seconds of triggering.
- The fundraiser progress bar shall update within 2 seconds after a successful transaction confirmation.
- The community feed shall display the latest posts and announcements in under 3 seconds using 10 Mbps Wi-Fi speed.
- RSVP responses shall be reflected in the attendee count within 2 seconds of submission.
- Notifications for upcoming or updated events shall be dispatched within 1 minute of event change.

4.4 Use Cases

This section presents detailed use cases of our system.

Figure 4.1 illustrates the UML Use Case Diagram for the Kaaravan application, representing how users interact with various system functionalities such as community management, event handling, donations, and messaging.

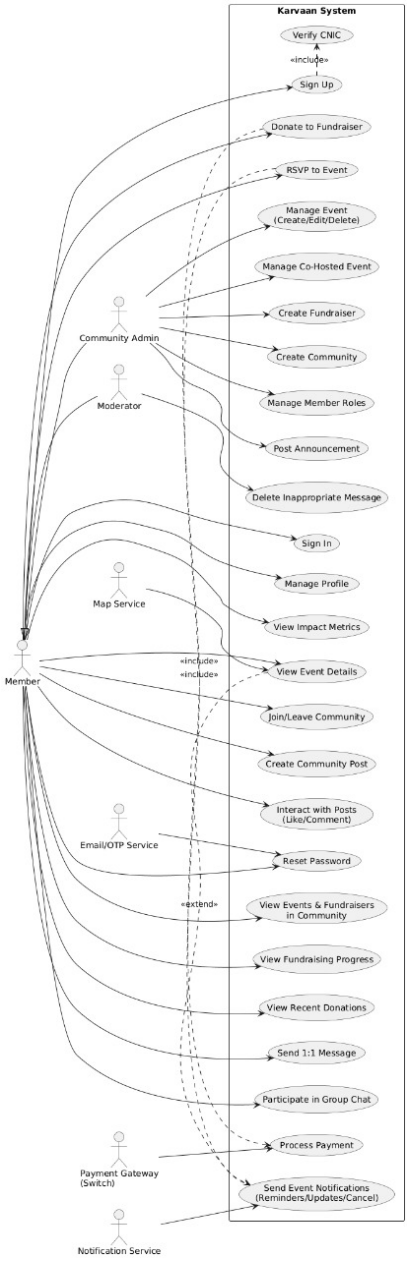

Figure 4.1: Karavaan Use Case Diagram

4.5 External Interfaces

This section describes the interfaces through which users and other systems interact with Karvaan.

4.5.1 Block Diagram

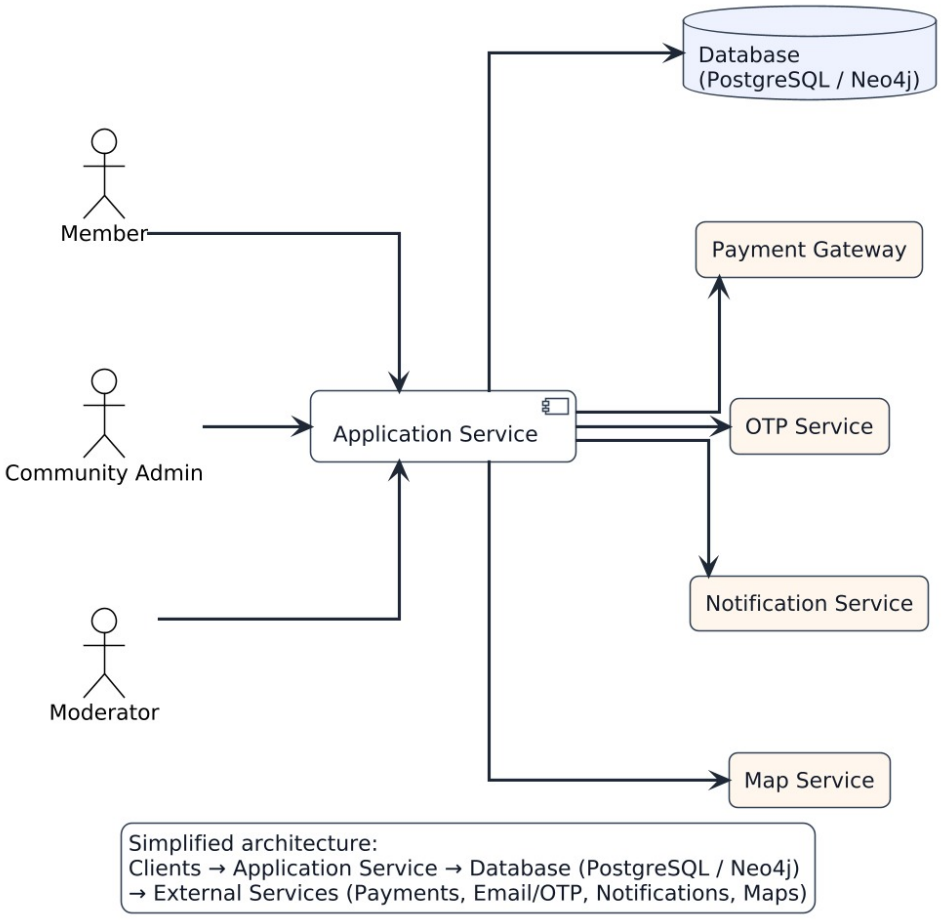

Figure 4.2: Block Diagram

Description: The block diagram illustrates the overall architecture and data flow of the Karvaan Application, highlighting how the main components of the system interact to deliver functionality. It begins with the User Interface (UI), where users

access the application through various screens such as login, communities, events, fundraisers, and profiles. These interfaces communicate with the Application Layer, which processes user inputs and manages interactions between different modules.

The Application Layer connects to a central database, which stores and retrieves data such as user information, event details, community data, and fundraiser records. This database ensures efficient data management, integrity, and scalability for the system.

Additionally, the system integrates with External Services such as Payment Gateways, Map APIs, and a Notification Service. These components handle secure payments, provide location-based event information, and deliver real-time updates to users.

Overall, the block diagram represents a modular and scalable architecture, where each component interacts seamlessly to support the platform's key functionalities while maintaining flexibility for future database and service integrations.

4.5.2 User Interfaces

This section includes our mockup screens and briefly explains them.

Figure 4.3: Landing Page

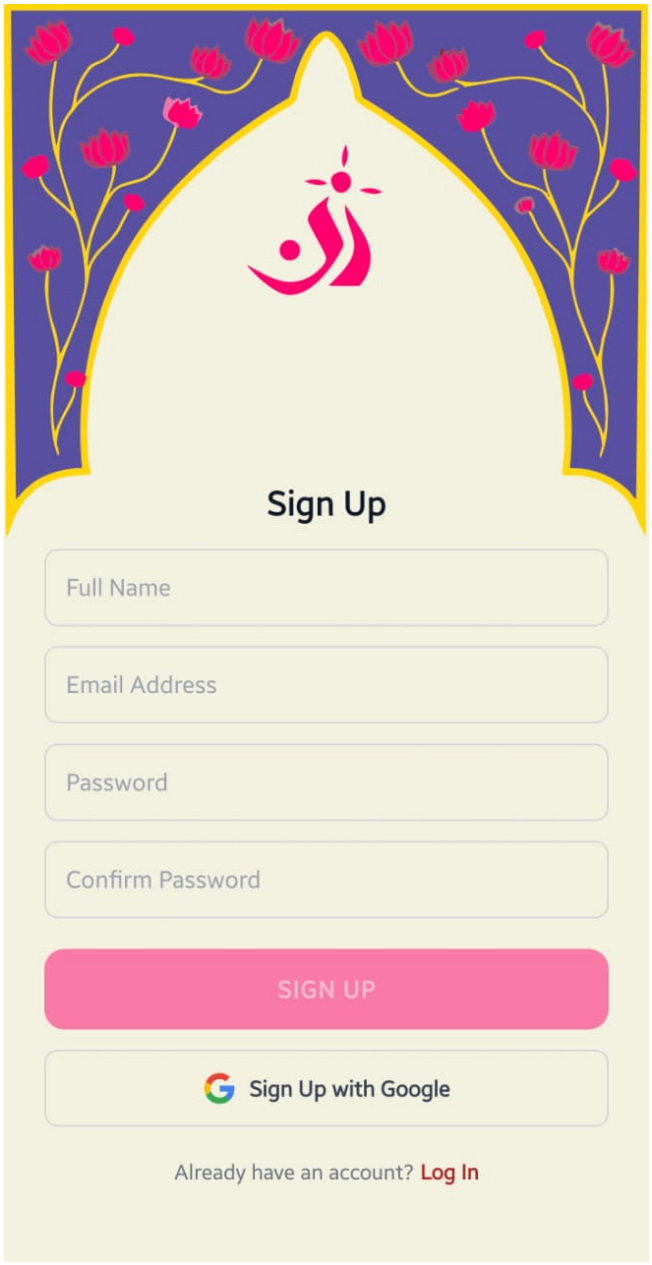

Figure 4.4: Sign up

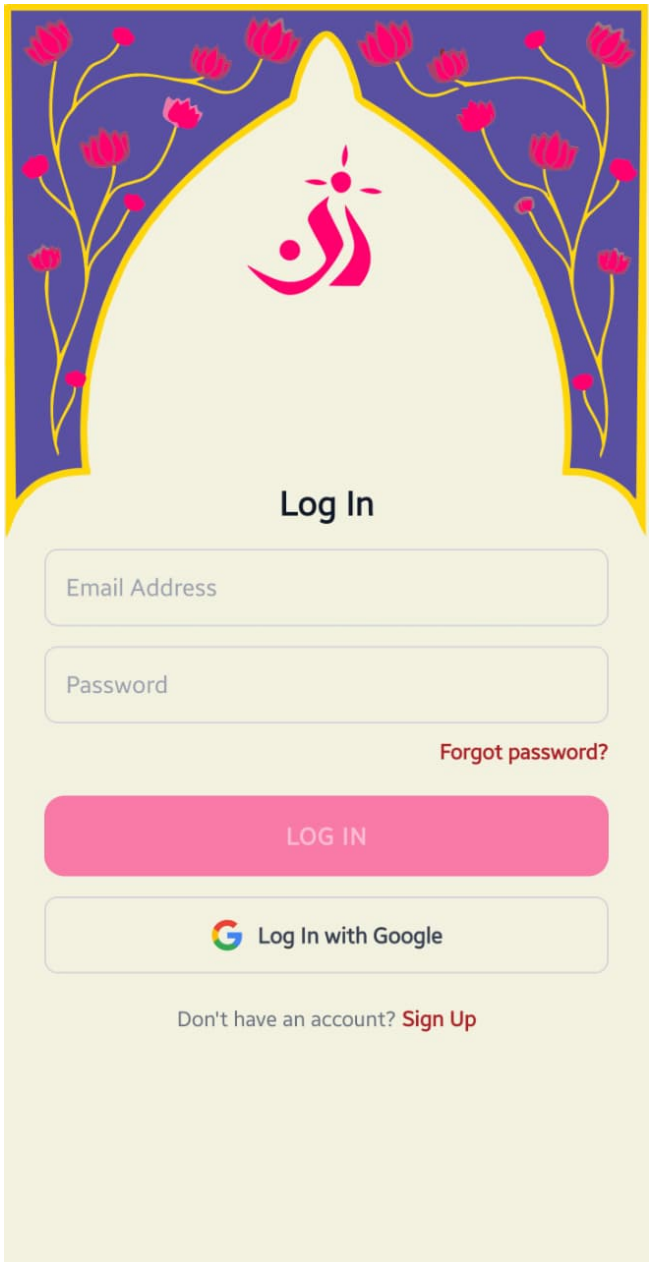

Figure 4.5: Sign in

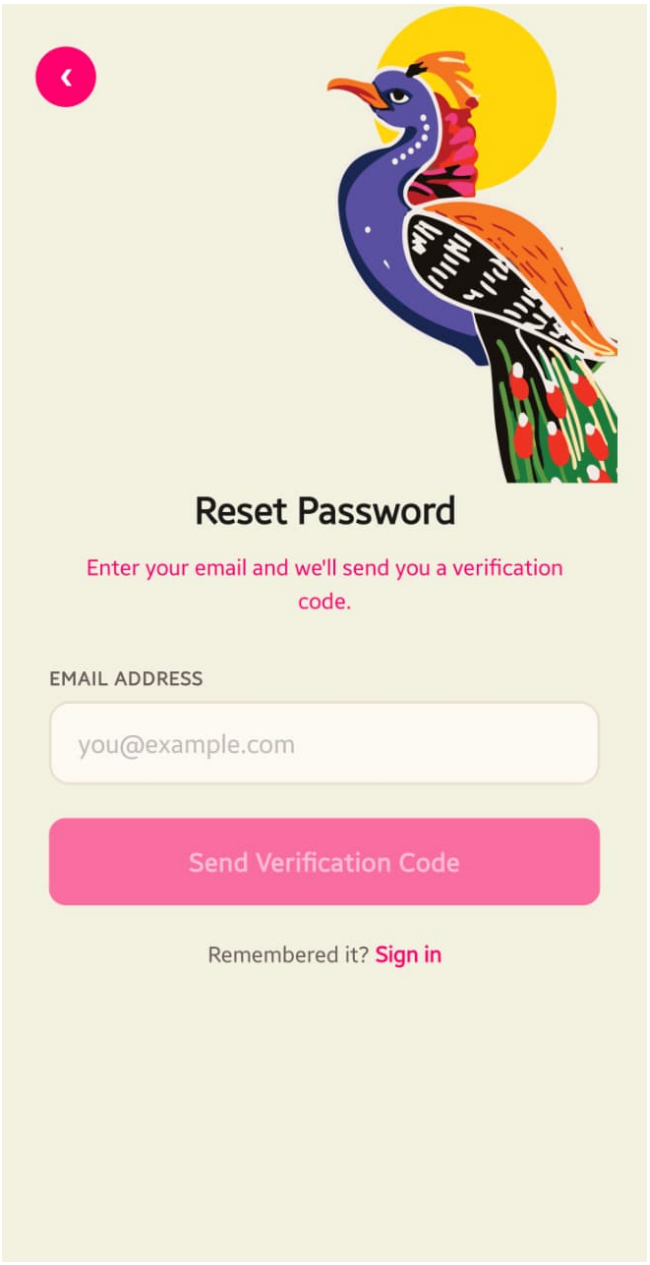

Figure 4.6: Forgot Password

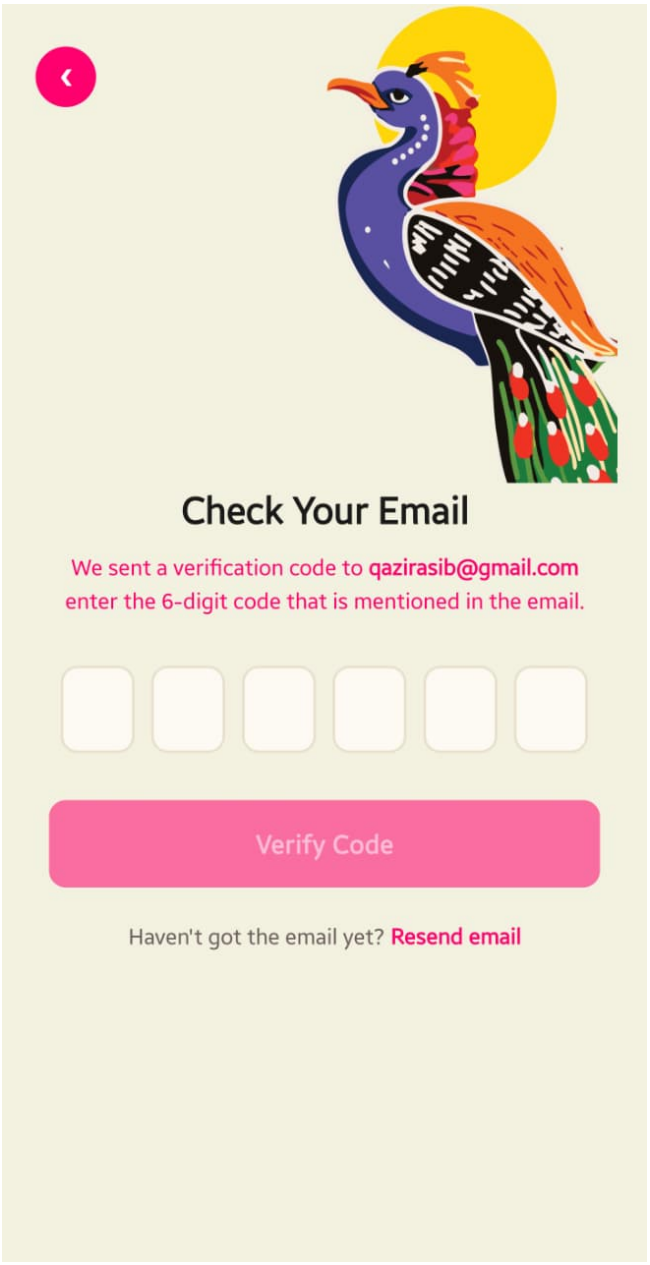

Figure 4.7: OTP

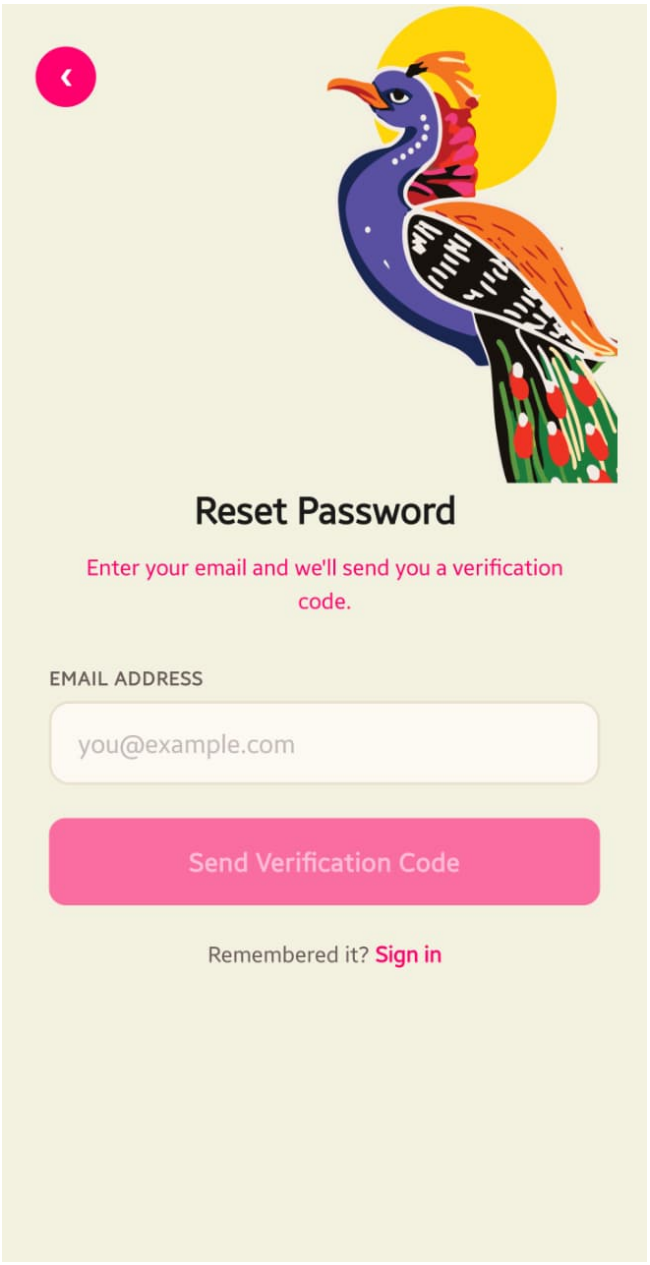

Figure 4.8: Reset Password

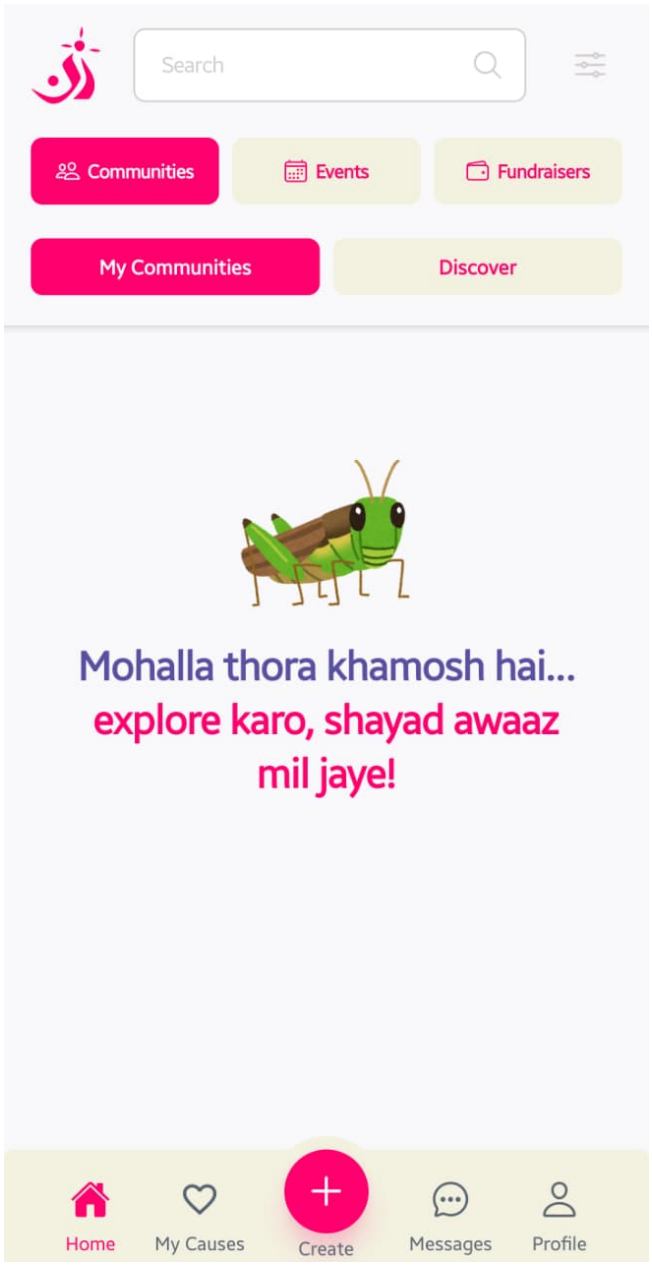

Figure 4.9: Home screen

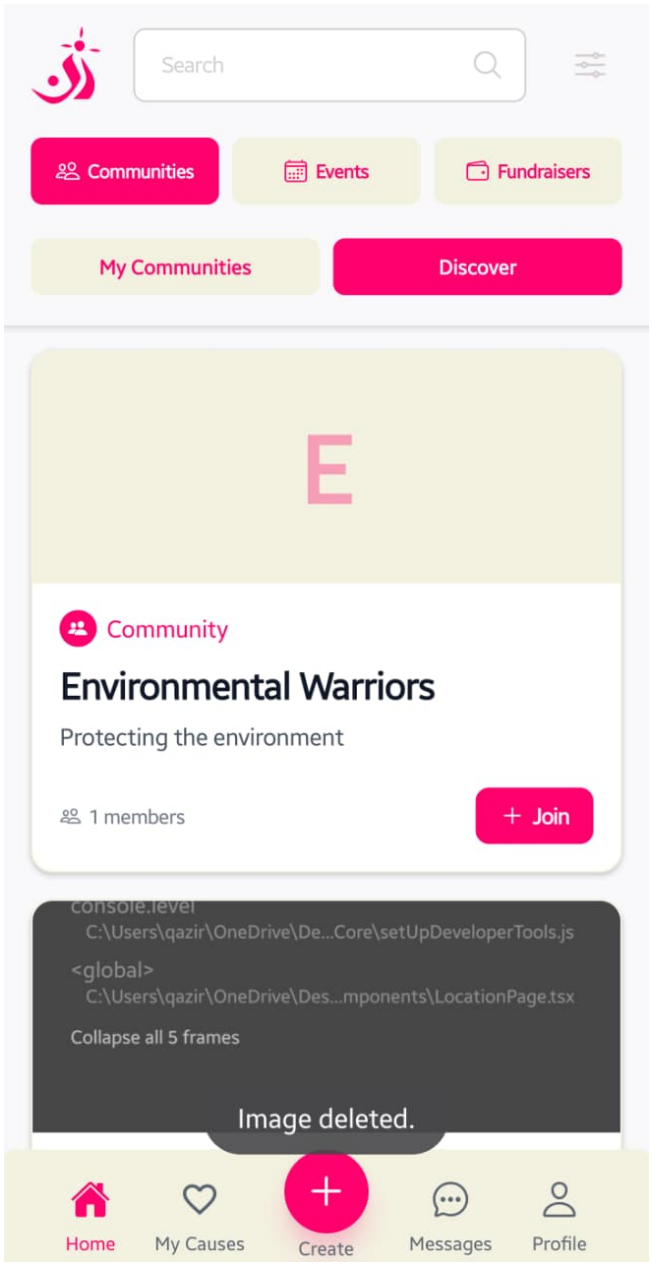

Figure 4.10: Discover Communities

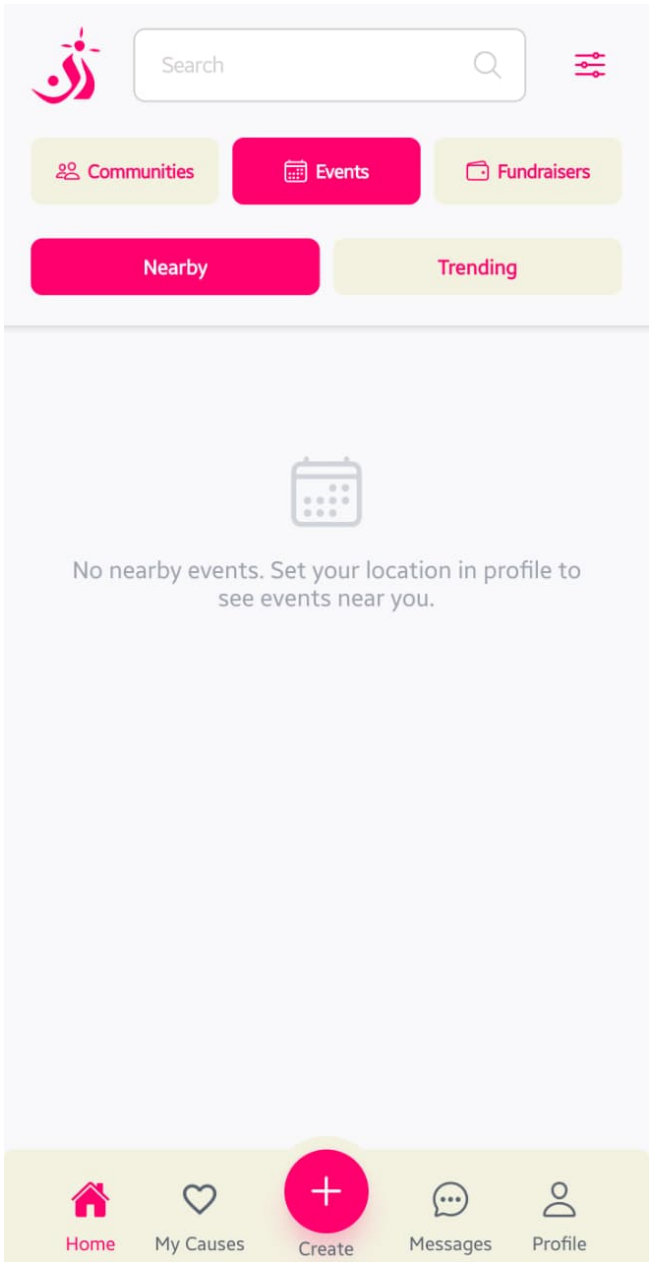

Figure 4.11: Nearby Events

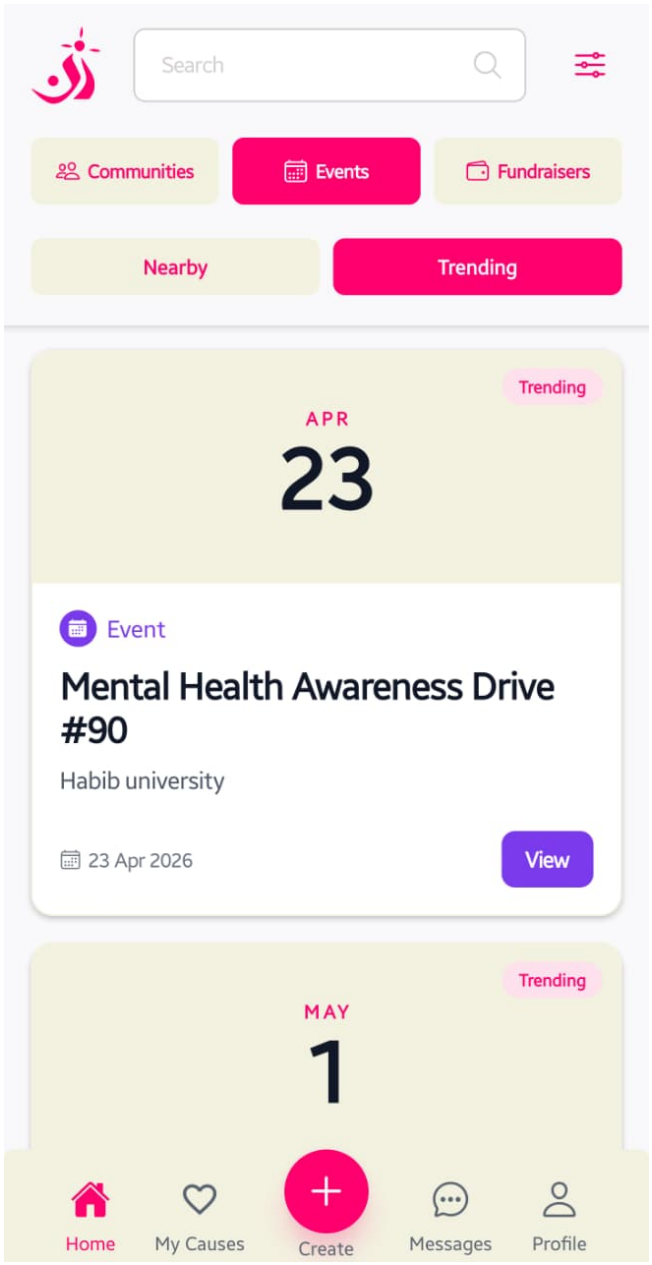

Figure 4.12: Trending Events

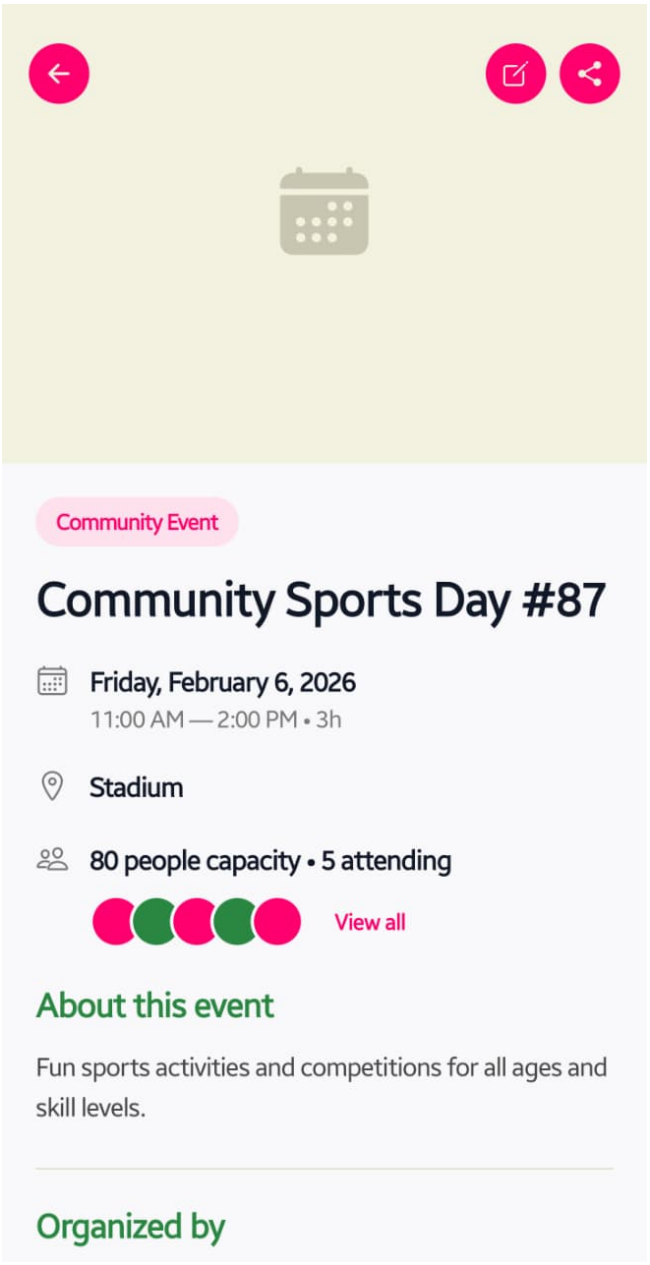

Figure 4.13: Event information

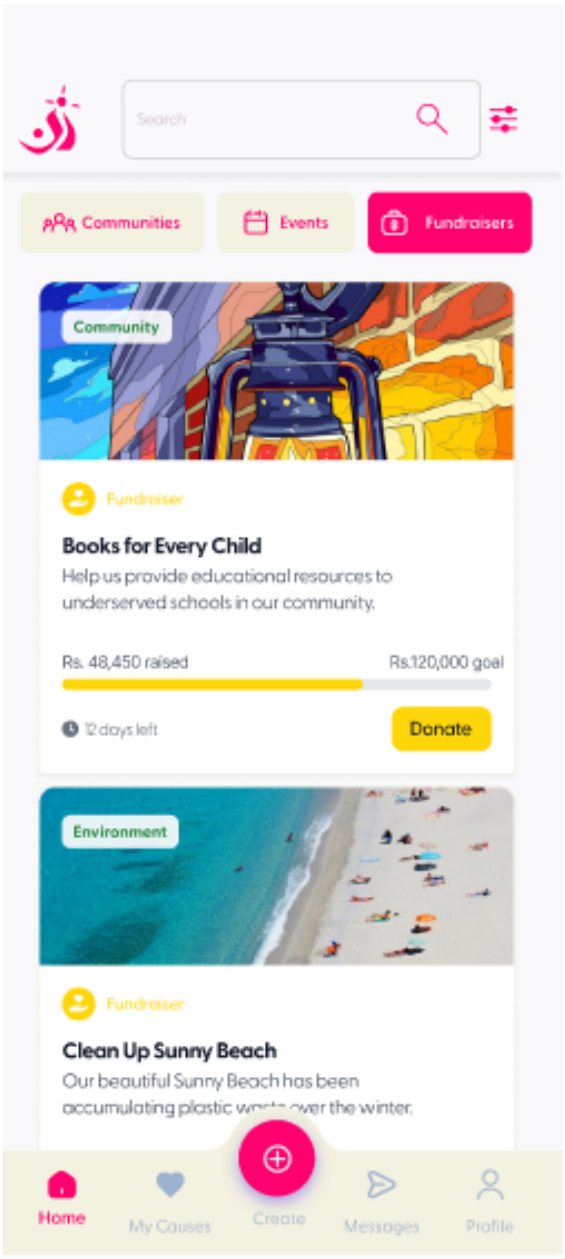

Figure 4.14: Fundraisers

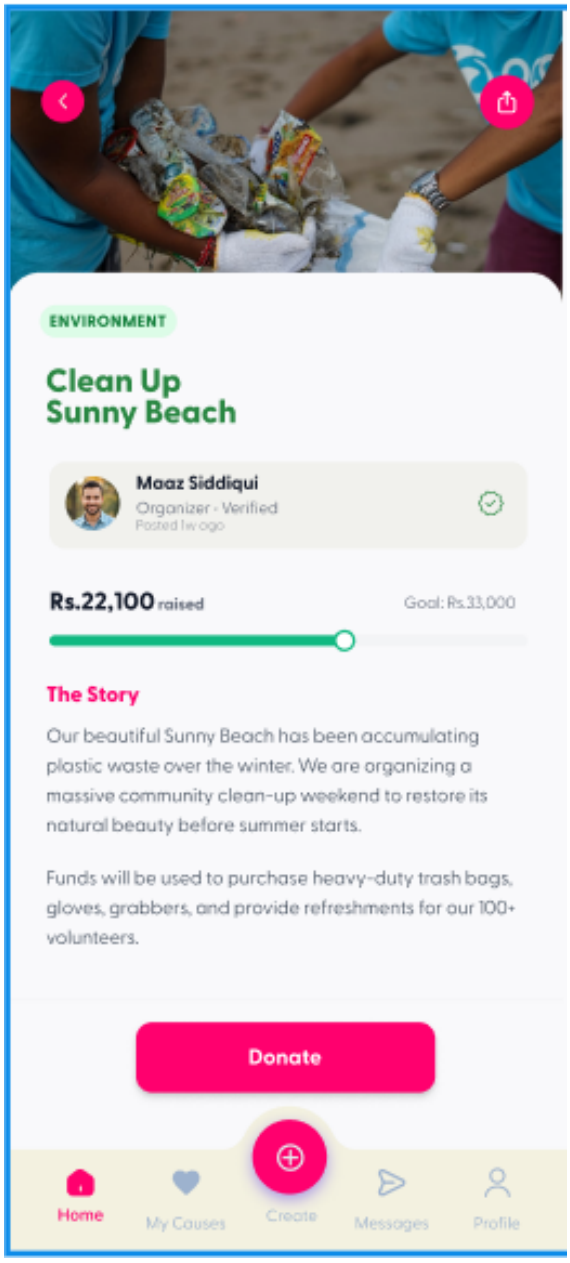

Figure 4.15: Fundraiser Information

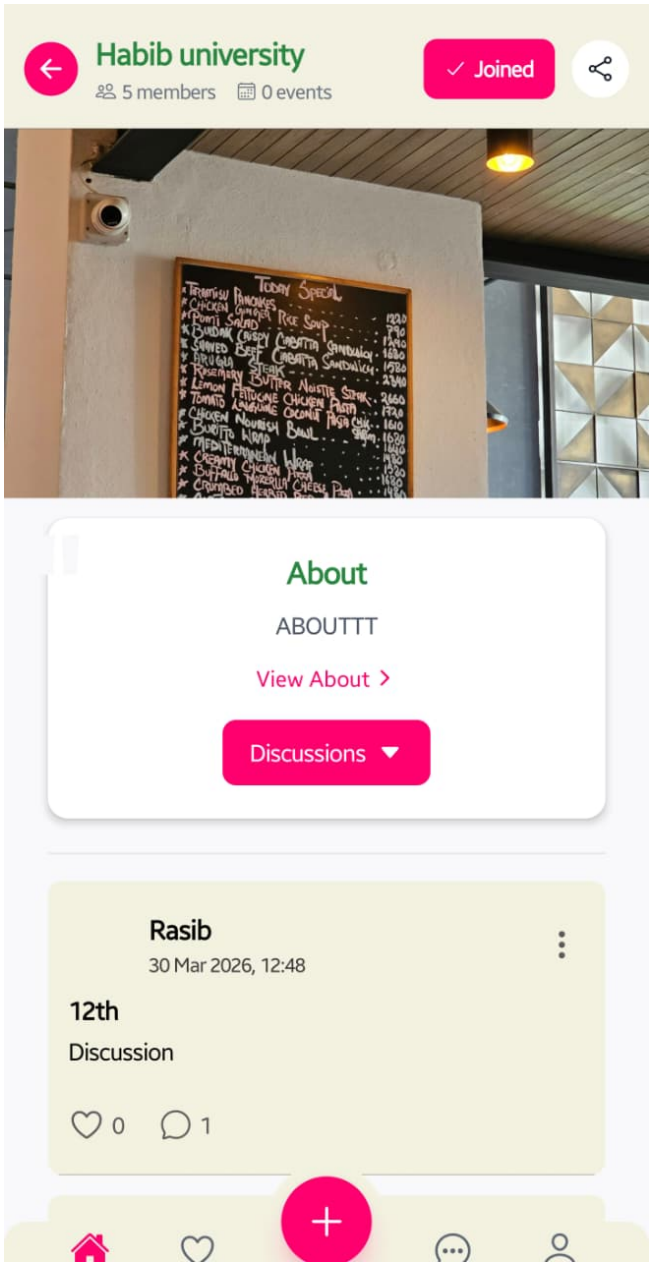

Figure 4.16: My Community Homepage

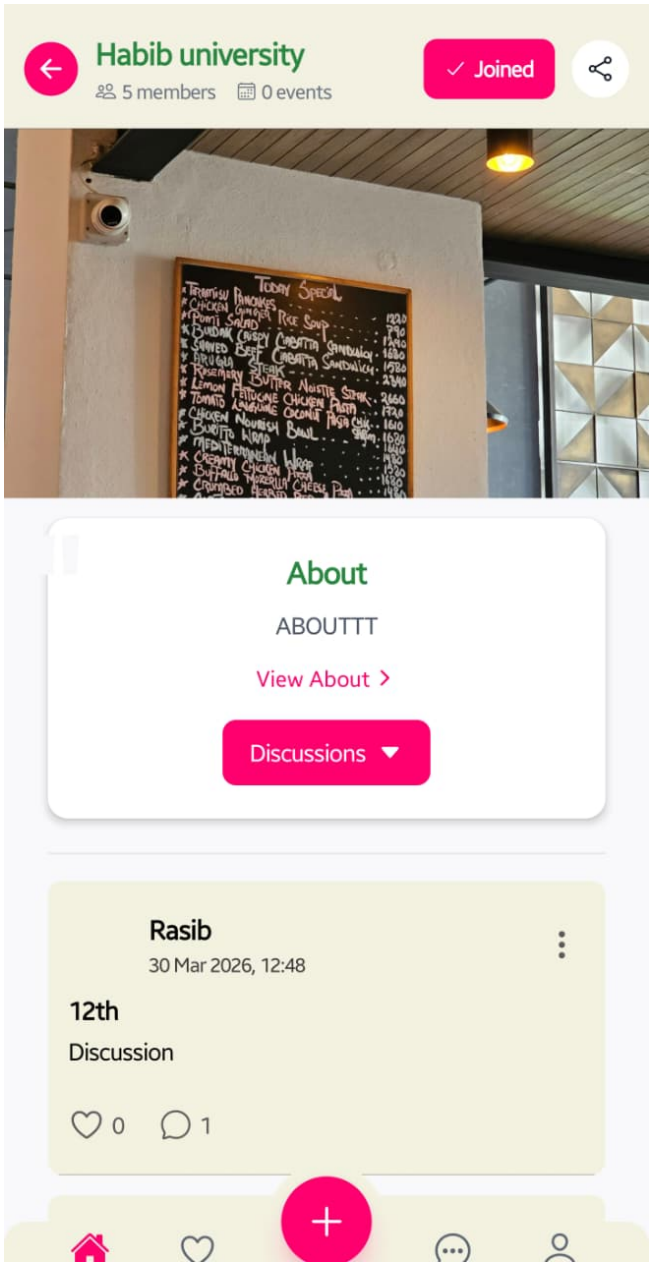

Figure 4.17: Community Discussion Page

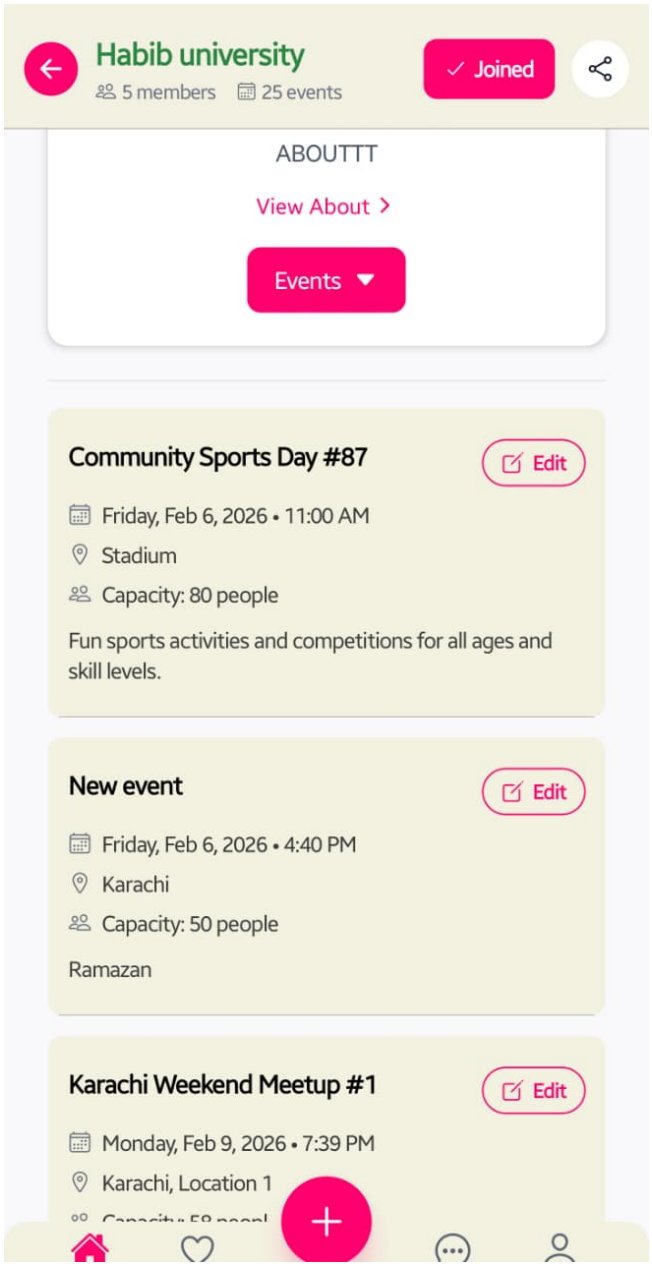

Figure 4.18: Community Event Page

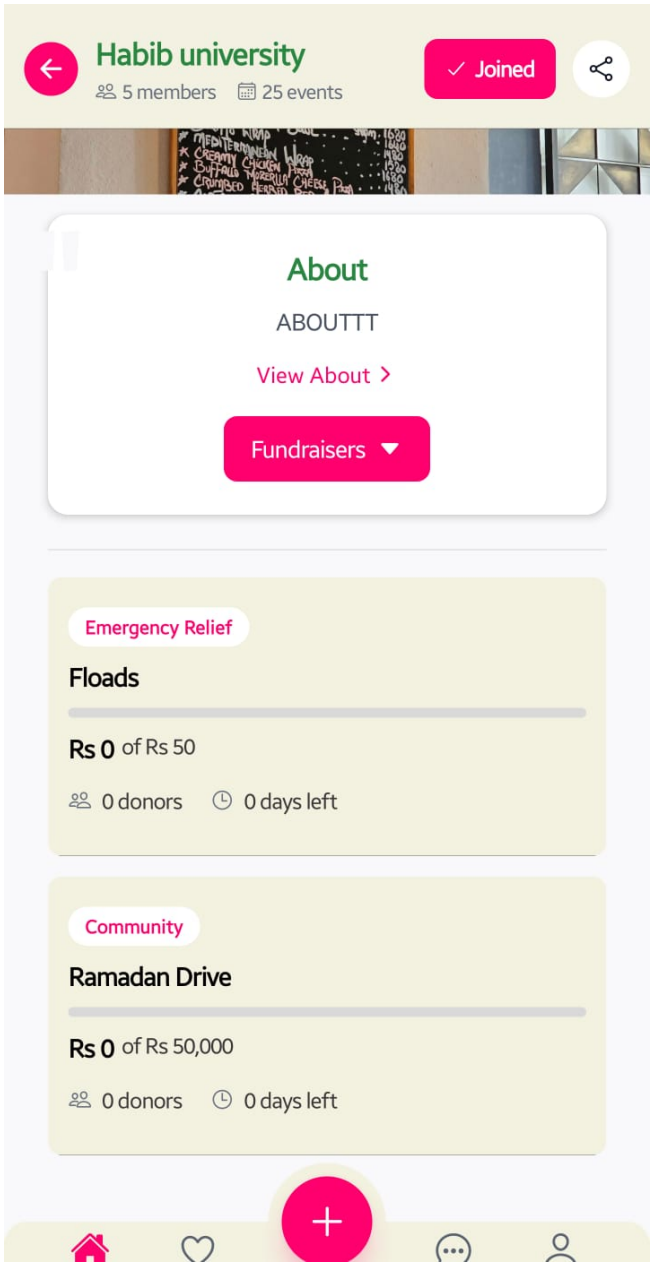

Figure 4.19: Community Announcement Page

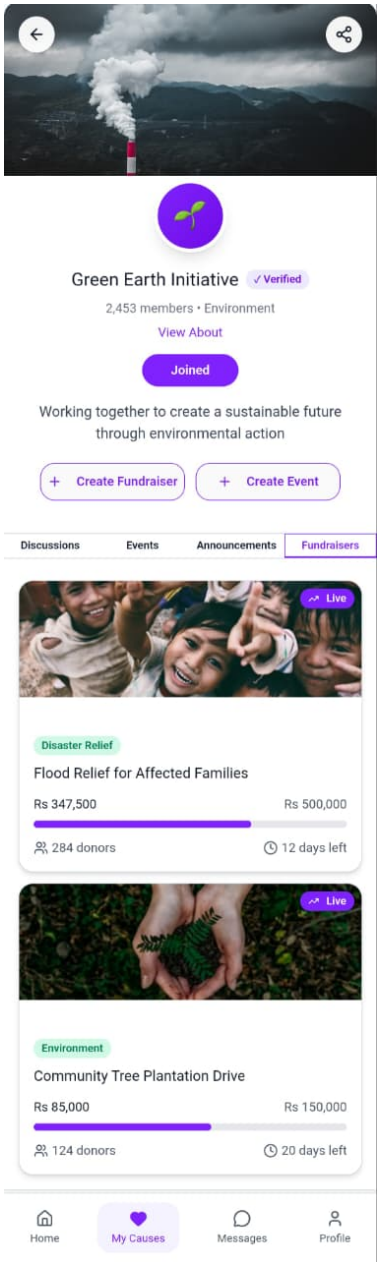

Figure 4.20: Community Fundraisers Page

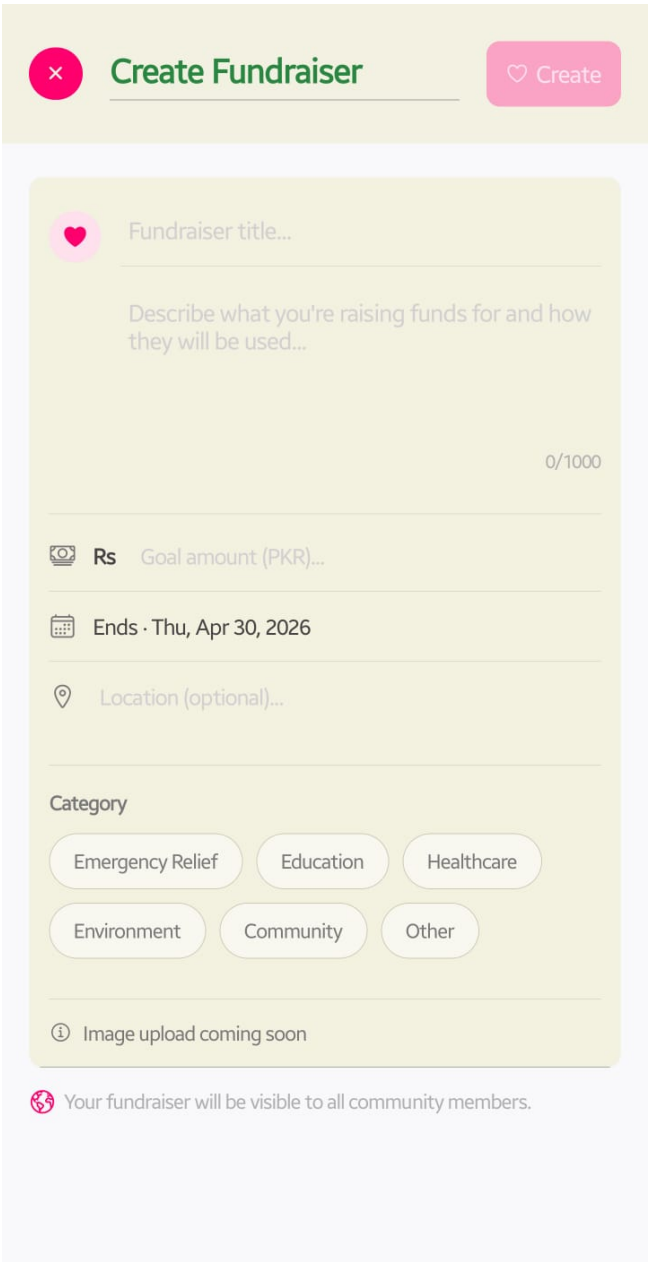

Figure 4.21: Create Fundraisers

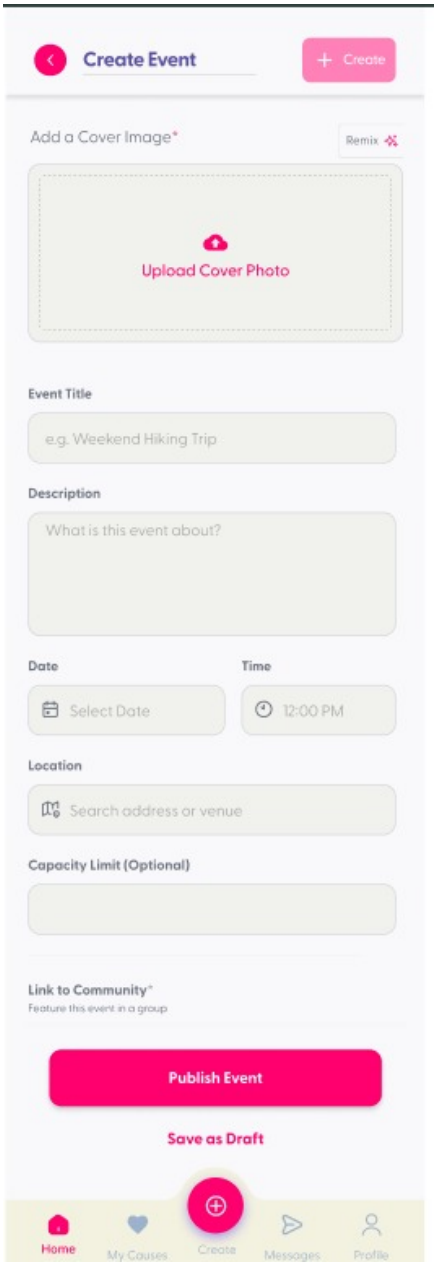

Figure 4.22: Create Event

Figure 4.23: Create Community

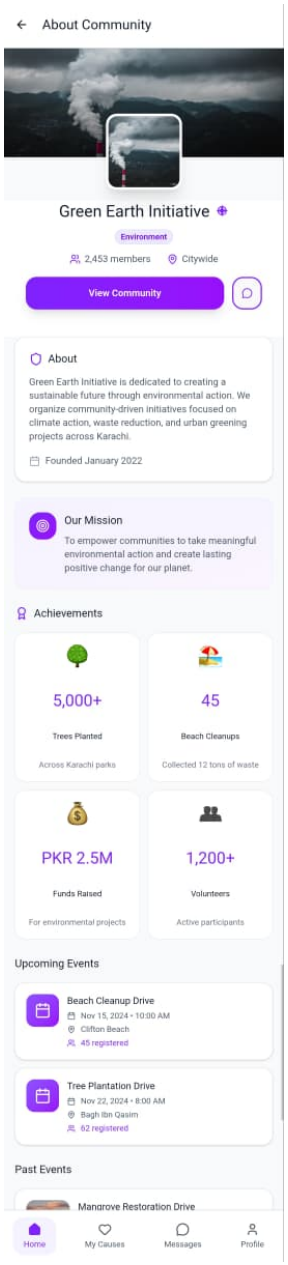

Figure 4.24: Community About Page

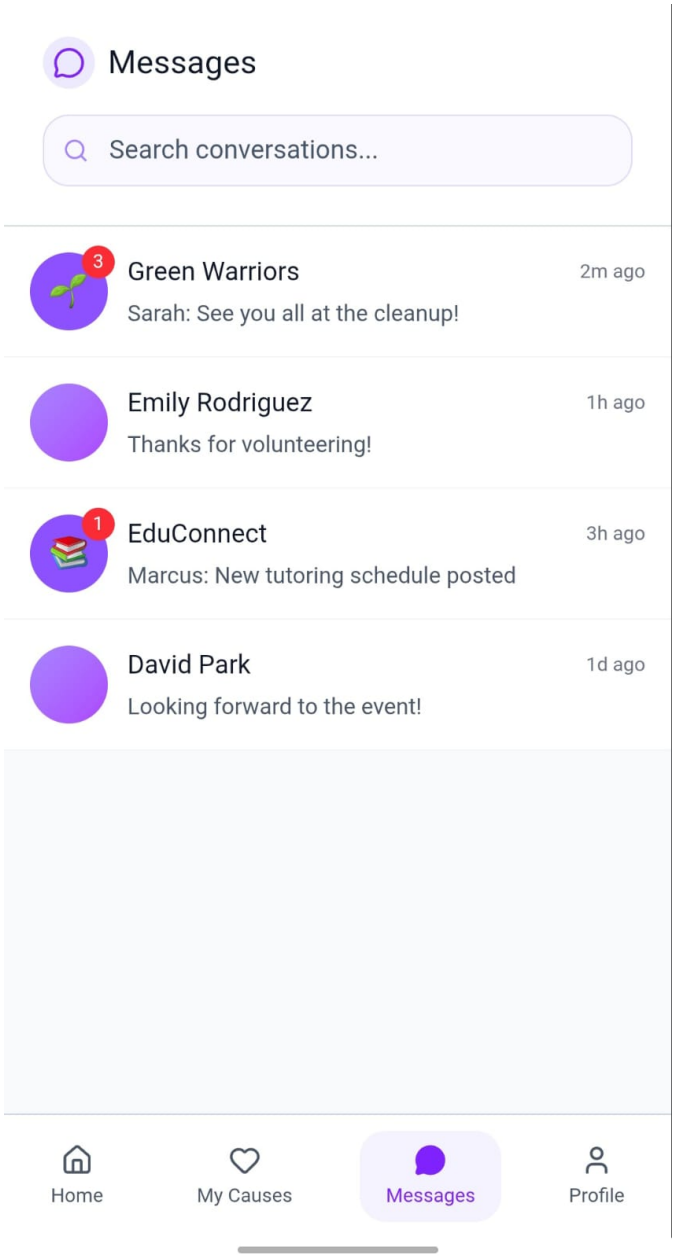

Figure 4.25: Conversation Page

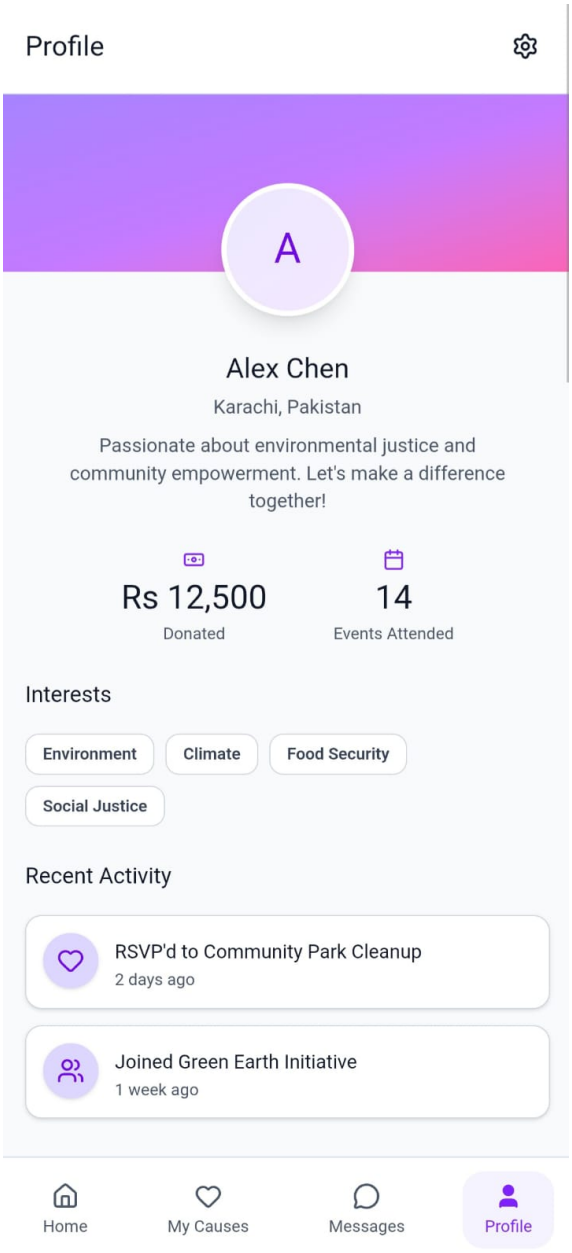

Figure 4.26: Profile Page

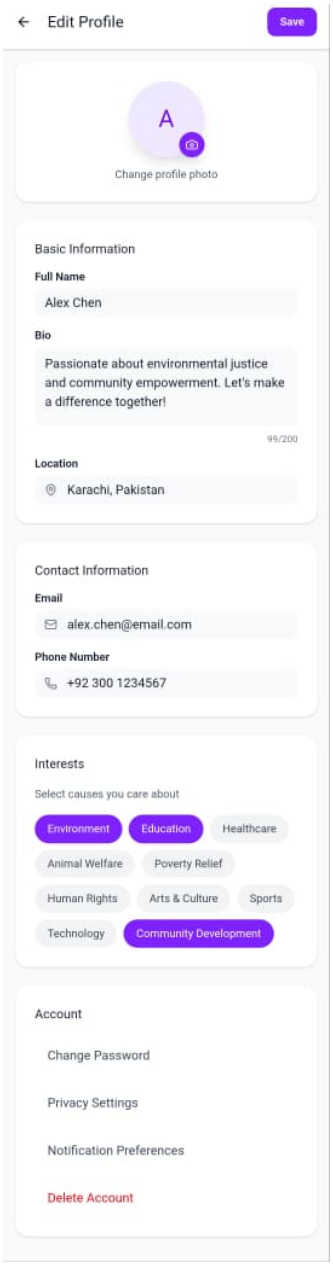

Figure 4.27: Edit Profile

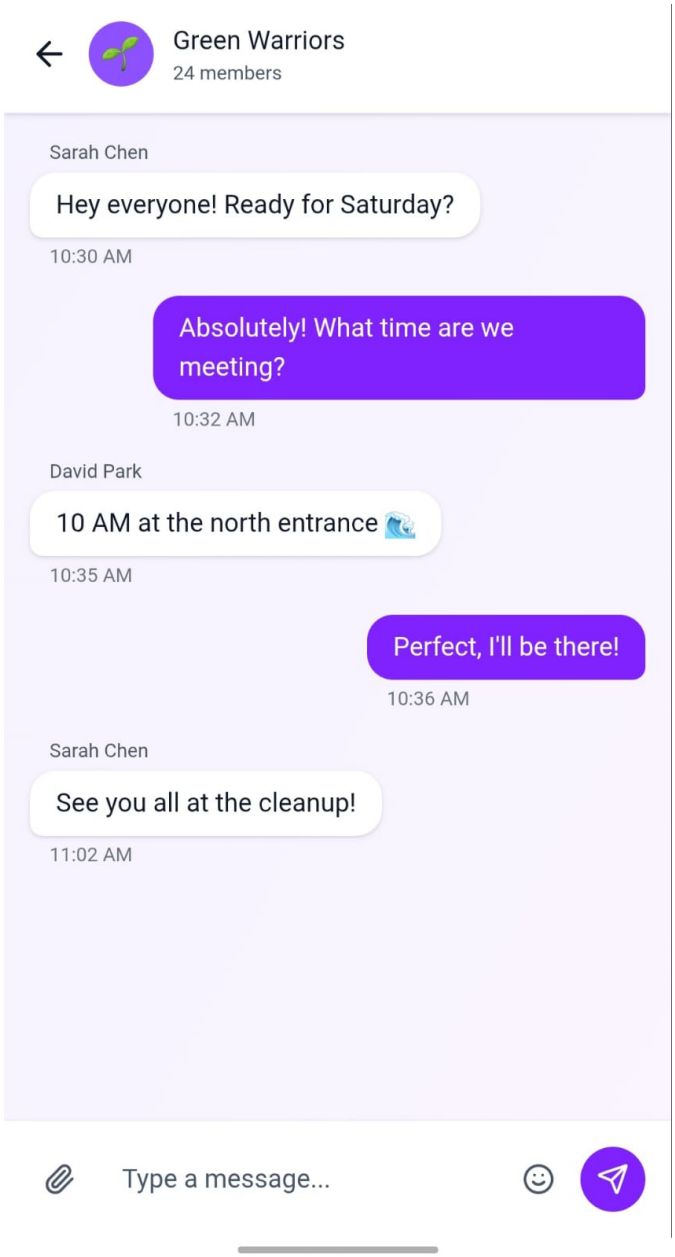

Figure 4.28: Messaging

4.5.3 Application Program Interface (API)

Kaarvan exposes a RESTful HTTP API built on Node.js and Express, using JSON for all request and response bodies. JWT-based authentication protects all routes, with tokens passed via the Authorization header; the same token is verified during the Socket.IO handshake for real-time connections. The API is organized into route groups covering user management (registration, login, Google OAuth, password recovery, profile, CNIC upload, location, and push tokens), communities (creation, discovery, membership, discussions, comments, likes, and moderation), events (full lifecycle, RSVP, and Recombee hooks), fundraisers (creation, donations, and proof uploads), messaging (direct and group messages with history), file uploads via UploadThing, and server-proxied Google location services. Real-time behavior — live message delivery, typing indicators, and room membership — is handled by Socket.IO over the same HTTP server, complementing the REST layer for low-latency flows.

4.5.4 Hardware/Communication Interfaces

Users interact with Kaarvan through smartphones or tablets running the Expo application over Wi-Fi or cellular. Communication uses HTTPS for REST and WebSockets for real-time chat, with TLS termination at the reverse proxy in production. The backend is a Node.js process connected to a hosted PostgreSQL database via Prisma. External integrations include UploadThing for file storage, Google APIs for location, Recombee for recommendations, and AWS SES for transactional email.

4.6 Datasets

Kaarvan's primary dataset is the operational data in PostgreSQL, covering users, communities, events, fundraisers, discussions, messages, notifications, and push tokens. A controlled interests vocabulary constrains user and community tagging. Event properties and user interactions are synced to Recombee for personalized recommendations. For demonstration purposes, the system was populated with synthetic seeded content to avoid exposing real user data.

4.7 System Diagram

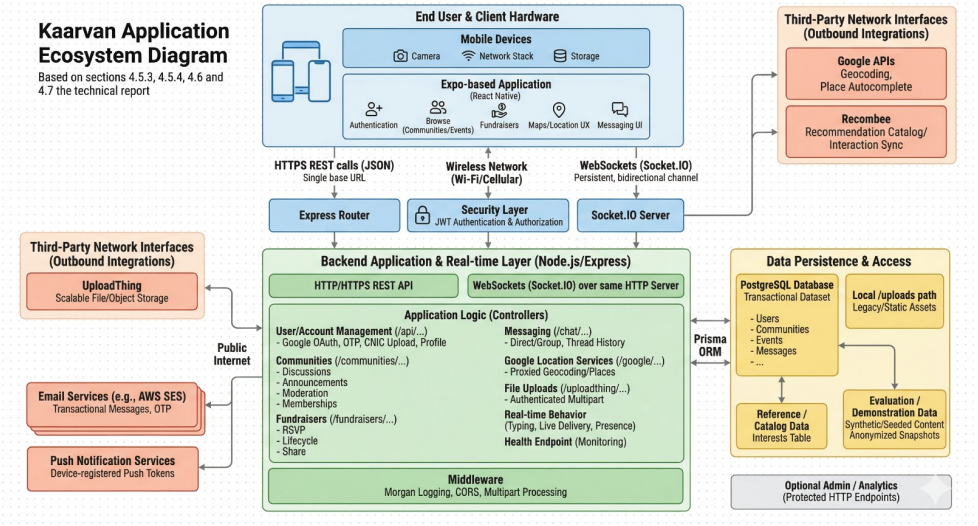

Figure 4.29: System Diagram

5. Software Design Specification (SDS)

This chapter provides important artifacts related to design of our project.

5.1 Software Design

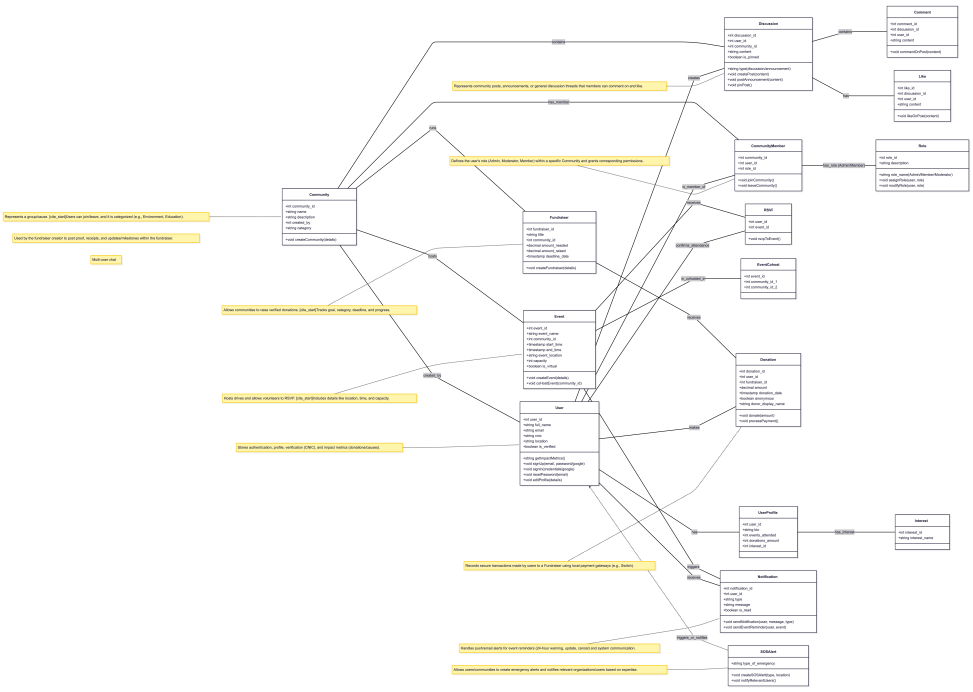

Figure 5.1: Class Diagram

The Karavaan system's architecture is built around several core entities that manage user interaction, community initiatives, and financial transactions. The central

component is the `User` class, which handles authentication, profiles, and verification through attributes such as `user_id`, `email`, and `cnic`. A User can sign up, sign in, or reset their password, and is linked 1-to-1 with a `UserProfile` to track impact metrics such as `events_attended` and `donations_amount`. Users belong to `Communities`, which are dedicated spaces for causes, and are managed through the `CommunityMember` class that defines their `role_id` (Admin, Moderator, Member) within the group. A `Community` acts as a hub for action, allowing creation of `Events` for volunteer activities and `Fundraisers` to raise money. An Event, with attributes like `start_time` and `event_location`, receives user participation via the `RSVP` class. A `Fundraiser` tracks the `amount_needed` and `amount_raised` and is supported by the `Donation` class, which records contributions from a User. To maintain transparency, organizers use `FundraiserUpdate` to post proof and receipts related to the use of funds. Communication occurs through `Discussion` posts within Communities, which users can respond to using `Comment`. Finally, the `Notification` class ensures timely updates such as event reminders, and the `SOSAlert` system handles emergency coordination.

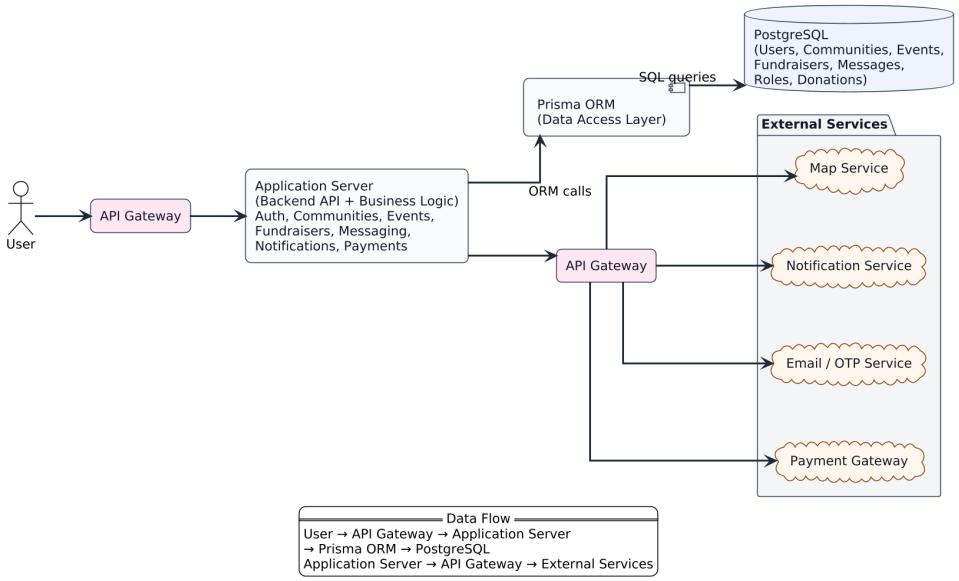

Figure 5.2: System Architecture Block Diagram

The following block diagram (Figure 5.2) illustrates Karavaan's high-level system architecture and how components interact to deliver platform functionality. The flow begins with the **User**, who accesses the system through the mobile or web interface.

All requests are routed through the **API Gateway**, which validates requests and forwards them to the **Application Server**. The Application Server contains the core business logic for authentication, communities, events, fundraisers, messaging, and notifications.

To manage data operations, the Application Server communicates with **Prisma ORM**, which acts as the data access layer. Prisma translates backend operations into SQL queries and interacts with the **PostgreSQL Database**, where all persistent data—such as users, communities, posts, events, and donations—is stored.

The system also integrates several **External Services** through a secondary API Gateway. These include the **Map Service** for location data, **Notification Service** for push alerts, **Email/OTP Service** for verification and password resets, and the **Payment Gateway** for donation processing. The architectural design is modular, scalable, and ensures that internal logic, data management, and third-party integrations remain cleanly separated.

5.2 Data Design

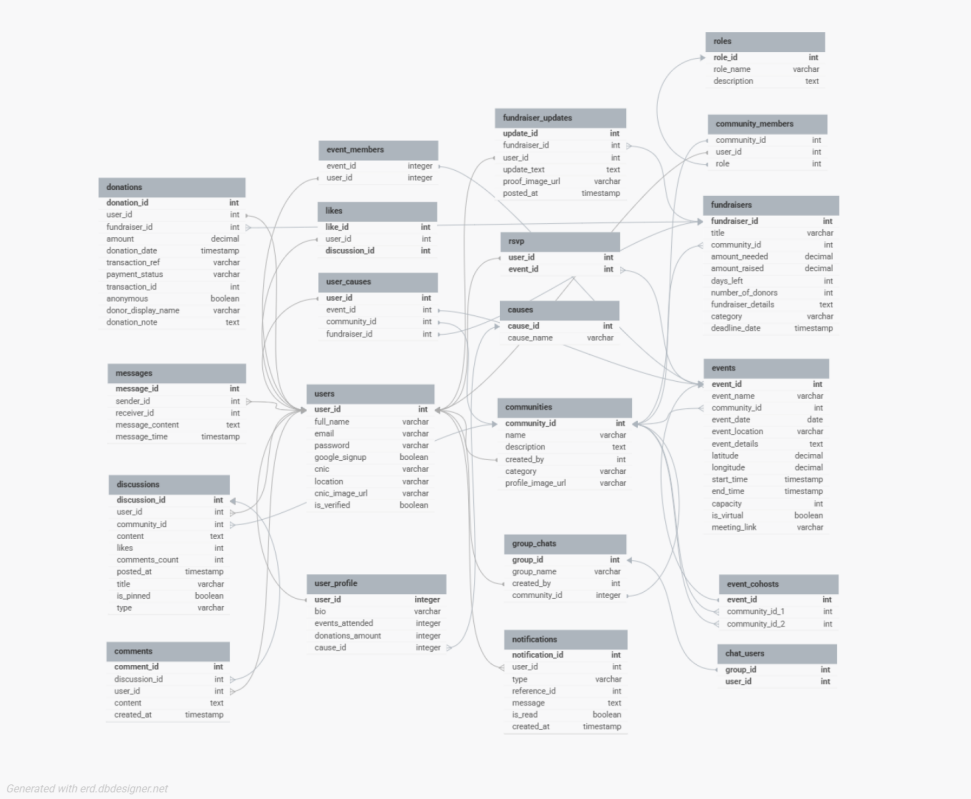

Figure 5.3: Database Schema

5.3 Technical Details

Overview:
Karavaan is implemented as a scalable, service-oriented platform that supports fundraising, community engagement, event creation, and real-time communication. The design combines a modular backend, a relational PostgreSQL database, and integrations with external services for payments, maps, notifications, and email/OTP. Figure 5.2 shows the high-level architecture, while Figure 5.3 presents the underlying database schema.

System Architecture

The request flow begins with the **User** who interacts with the mobile or web client. All calls are routed to the **API Gateway**, which acts as a single entry point. It performs basic request validation, authentication checks, and rate limiting before forwarding traffic to the internal services.

The **Application Server** hosts the core backend API and business logic. It contains modules for:

- Authentication and user management
- Communities, discussions, and member roles
- Events, RSVPs, and volunteer coordination
- Fundraisers, donations, and organizer updates
- Real-time messaging and notifications

For all data operations, the Application Server communicates with **Prisma ORM**, which serves as the data access layer. Prisma translates high-level model operations into SQL queries and manages schema migrations. It also provides type-safe access to the database, reducing the likelihood of runtime errors and SQL injection vulnerabilities.

The persistent data is stored in a **PostgreSQL** database. PostgreSQL is chosen for its ACID guarantees, strong relational model, and support for indexing and complex joins. The database stores users, communities, events, fundraisers, donations, messages, roles, notifications, and related entities, as illustrated in Figure 5.3.

The system also integrates multiple **External Services** through a secondary API gateway. These include:

- **Map Service** for event and community location data.
- **Notification Service** (e.g. FCM/APNs) for push notifications and alerts.
- **Email/OTP Service** for account verification and password reset flows.
- **Payment Gateway** for secure processing of donations and transaction callbacks.

Data Flow

The main data flow for typical user interactions can be summarized as:

1. User sends a request from the app to the API Gateway.
2. API Gateway authenticates and routes the request to the appropriate module on the Application Server.
3. Application Server executes business logic and interacts with Prisma ORM.
4. Prisma ORM issues SQL queries to PostgreSQL and returns the results to the Application Server.
5. When required, the Application Server calls external services (maps, payments, notifications, email/OTP) through the external API Gateway.
6. Responses are returned to the user via the original API Gateway.

This layered flow provides loose coupling between components and allows each part of the system (backend, database, and third-party integrations) to scale independently.

Database Design and Entity Relationships

Karavaan uses a normalized relational schema to capture the relationships between users, communities, events, and fundraisers.

- The `users` table stores core user information such as authentication data, contact details, and verification status. Each user has a 1-to-1 relationship with `user_profile`, which keeps impact metrics like `events_attended` and `donations_amount`.
- `communities` represent cause-based groups. Membership is managed via `community_members`, which links users to communities and assigns a `role_id` from the `roles` table (e.g. Admin, Moderator, Member).
- `events` are linked to communities and store attributes such as date, time, location (including latitude/longitude), capacity, and whether the event is virtual. User participation is captured through `rsvp` and `event_members`.

- `fundraisers` also belong to communities and track fields such as `amount_needed`, `amount_raised`, `deadline_date`, and `category`. Actual monetary contributions are stored in the `donations` table, which references both the donor and the fundraiser and supports anonymous donations and donor notes.
- Transparency is maintained via `fundraiser_updates`, where organizers upload proof, receipts, and progress updates for donors.
- Community interaction is supported through `discussions`, `comments`, `likes`, and `group_chats`, enabling both threaded conversations and real-time chat.
- The `notifications` table records system notifications sent to users (e.g. new event, donation confirmation, role changes) and tracks whether each notification has been read.

This schema enables queries such as:

- retrieving all events for communities a user has joined,
- aggregating a user's total donation impact across different fundraisers,
- filtering fundraisers by community, category, or location,
- generating personalized feeds of local events and active fundraisers.

Real-Time Messaging

Real-time messaging in Karavaan is implemented using the WebSocket protocol. Each connected client maintains a persistent WebSocket connection to the server, enabling:

- low-latency, bi-directional communication,
- group conversations mapped to `group_chats` and chat rooms,
- delivery and read-status indicators,
- synchronization of message history stored in the `messages` table.

WebSockets are preferred over traditional HTTP polling because they avoid repeated connection overhead and scale better under concurrent usage.

Scalability, Performance, and Security

To support future growth and high user load, the architecture incorporates:

- **Horizontal scaling** of the Application Server instances behind the API Gateway.
- **Caching** of frequently accessed data (e.g. trending events, community feeds, basic user profiles) using an in-memory store such as Redis.
- **Database optimization** via indexing, query tuning, and potential read replicas for analytics-heavy workloads.
- **Rate limiting** and abuse prevention at the API Gateway to restrict excessive OTP requests, login attempts, or donation spam.

Security is enforced through:

- JWT-based authentication and role-based access control,
- encrypted password storage using secure hashing algorithms,
- HTTPS for all client–server communication,
- secure integration with the payment gateway using TLS and signed callbacks,
- validation and sanitization of all user input, with Prisma ORM providing an additional safety layer for database queries.

Overall, these technical design choices ensure that Karavaan is robust, extensible, and capable of supporting a transparent and trustworthy social-impact ecosystem.

6. Methodology, Experiments and Results OR Testing and Analysis

6.1 System Modules

The Kaarvan platform is structured around several key modules that together enable community organization, collaboration, and fundraising.

6.1.1 Community Management

The community module allows users to create and join communities based on shared interests or causes. Community administrators can manage members, publish announcements, create events, and launch fundraisers. This module forms the central organizational structure of the platform and acts as a hub for discussions, updates, and activities.

6.1.2 Event Management

The event management module enables communities to organize activities such as volunteer drives, awareness campaigns, or workshops. Administrators can create events by specifying details such as event title, description, location, date, and participant capacity. Users can RSVP to events and view upcoming activities through the platform's event feed.

6.1.3 Fundraising System

The fundraising module allows communities to create campaigns to support specific causes. Each fundraiser contains a target amount, description, deadline, and progress indicator. Users can view fundraising campaigns and track progress toward the target goal. This feature improves transparency and helps communities coordinate financial support for initiatives.

6.1.4 Messaging and Discussions

Communication between users is supported through two mechanisms. Threaded discussions allow community members to post updates and comments related to community activities, while the real-time messaging system enables instant communication between users. The messaging functionality is implemented using Socket.io to provide low-latency communication.

6.2 Trending Event Ranking Algorithm

The trending events module ranks upcoming events based on user engagement rather than simple insertion or creation order. Engagement is defined through user–item interactions, specifically event detail views (when a user opens an event page) and RSVP actions (when a user commits to attending). These interaction signals are continuously logged and used to inform the ranking process, enabling the system to distinguish between events that merely attract attention and those that successfully convert interest into participation.

The event catalog is not limited to organically created in-app listings. For the purposes of this project, a significant portion of events is manually curated from external public sources, such as Facebook event pages and similar platforms. These listings are normalized and stored within the Kaarvan database with standardized attributes, including a stable event ID, title, description, schedule, venue information, modality (physical or virtual), geographic coordinates (when available), and an associated host community. This curated dataset serves as the ground-truth catalog, while the recommender system operates on a structured projection of this data.

Kaarvan integrates Recombee, a third-party recommender-as-a-service. Each event in the database is mapped to a corresponding item within Recombee, and relevant item properties are synchronized via its API. These properties include attributes that support filtering and learning, such as community identifiers, event modality, timestamps, location-based features, and dynamically updated RSVP counts. User

interactions (views and RSVPs) are streamed to Recombee in real time. A predefined scenario within the Recombee project governs the behavior of the "trending" request. While the internal learning mechanisms of Recombee remain abstracted, the effectiveness of the system depends on the quality and structure of the input data, including event attributes and interaction signals.

Upon receiving a ranked list of event IDs from Recombee, the Kaarvan backend retrieves the corresponding records from its database. A post-processing step ensures that only upcoming events are included in the final feed by filtering out past events based on their scheduled time. This guarantees that temporal validity is enforced by the authoritative database rather than relying solely on the recommender's last synchronization state.

To ensure robustness, a fallback mechanism is implemented in case of API failure or insufficient recommendations. In such scenarios, the system performs a SQL-based reranking over the same curated dataset. Events are filtered using the same constraints (e.g., upcoming-only), then sorted by descending RSVP count (popularity) and ascending event date (temporal proximity). The results are merged with any available recommender outputs and deduplicated to produce the final list. This fallback strategy ensures continuity of service while still leveraging live engagement metrics.

6.3 Testing and Evaluation

To ensure the reliability and functionality of the Kaarvan platform, testing was conducted throughout the development lifecycle using a combination of structured acceptance criteria and manual exploratory testing across both the mobile application and backend API.

6.3.1 Functional Testing

Functional testing was performed against a formal set of user stories and acceptance criteria documented for the platform's core modules. The authentication module alone covered 23 acceptance criteria for login and 20 for signup, validating flows such as email/password authentication, Google OAuth, forgot password, OTP verification, and password reset. Each criterion defined explicit preconditions, expected outcomes, and test steps — for example, verifying that the login button enters a disabled state with 0.7 opacity during an active request, that email inputs are trimmed of whitespace before submission, and that Google Sign-In cancellations fail silently without surfacing an error alert.

Testing was conducted manually by exercising each flow on Android through the Expo development build. Edge cases including empty field submission, mismatched passwords, invalid OTP formats, expired reset tokens, and network disconnection scenarios were all tested explicitly. Client-side validation logic — such as real-time password mismatch detection and numeric-only OTP filtering — was verified by direct interaction. Navigation behavior was also validated, including the use of `router.replace()` after password reset to prevent users from navigating back into the reset flow.

Beyond authentication, functional testing extended to the community features throughout development, covering discussions, comments, likes, events, fundraisers, member management, and direct messaging. Testing here was primarily exploratory — features were exercised as they were built and integrated, with issues identified and resolved within the same development cycle.

6.3.2 Integration Testing

Integration testing focused on verifying the contract between the React Native frontend and the Node.js/Express/PostgreSQL backend. Each API endpoint was tested against its expected request and response schema. For instance, the `POST /api/login` endpoint was verified to return a token on valid credentials, respond with a 401 status on invalid credentials, and surface a meaningful error message the client could display. Similar validation was performed across the registration, Google OAuth, OTP, and password reset endpoints.

Several integration issues were identified and resolved during development. A notable example was the real-time messaging feature built on Socket.io, where messages sent by the current user were not appearing in the chat view due to a stale closure bug — the socket event listener was capturing an outdated reference to the message list state. This was resolved by restructuring the listener to always reference the latest state. Another class of issues involved profile images not rendering consistently across screens; this was traced to inconsistent handling of image URL construction from the backend response and was fixed by normalizing image URL resolution at the API service layer. Features that were missing or incomplete were also identified through end-to-end flow testing, as gaps became apparent when navigating through the app as a real user rather than testing components in isolation.

Backend integration was additionally validated using manual API requests during development to confirm endpoint behavior before wiring up the frontend, reducing the surface area of bugs introduced at the integration layer.

6.3.3 Performance Observation

Formal performance benchmarking was not conducted; however, performance was observed qualitatively throughout development and testing on physical devices. The application's responsiveness was found to be dependent on network conditions. On stable Wi-Fi connections, core flows such as login, feed loading, and navigation between tabs performed acceptably. On slower connections, latency became noticeable, particularly on the profile screen and in data-heavy views such as the events and fundraisers listings.

To address this, pagination was implemented on feed-style screens to limit the volume of data fetched per request, which improved perceived load times. Optimistic UI patterns were applied in places such as comment likes to ensure interactions felt immediate regardless of network latency. Push notifications were delivered via Firebase Cloud Messaging through an EAS production build, which performed reliably in testing. Real-time messaging latency over Socket.io was acceptable under normal Wi-Fi conditions, though no stress testing was performed under concurrent load.

6.4 Summary

Kaarvan was developed and evaluated through a combination of structured functional testing, integration verification, and continuous manual testing throughout the development lifecycle. A comprehensive set of user stories and acceptance criteria — covering authentication, navigation, validation, and error handling — provided a consistent baseline against which feature correctness was assessed. Integration testing surfaced and resolved meaningful issues including real-time messaging state bugs and inconsistent profile image rendering. Performance, while not formally benchmarked, was monitored qualitatively and addressed through pagination and optimistic UI strategies. The platform was found to be functionally stable across its primary user flows on both Android and iOS under normal network conditions.

7. Conclusion and Future Work

Future Work

Future work will focus on transitioning Kaarvan from a functional prototype to a fully deployed and scalable production system. The immediate priority is to publish the mobile application on platforms such as the Google Play Store and Apple App Store. This will enable real-world usage and provide valuable insights through user feedback and analytics.

A key next step is comprehensive user testing and evaluation. Structured usability studies and performance testing will be conducted to identify bottlenecks, improve system reliability, and refine the overall user experience. Feedback from real users will guide iterative improvements in both functionality and interface design.

From a product perspective, the user interface will be further enhanced by introducing a consistent design system, improving accessibility, and optimizing navigation flows. This will ensure a more intuitive and seamless interaction for users across different devices and use cases.

On the technical side, the integration of a secure and fully functional payment gateway will be a major enhancement. Enabling real financial transactions within the platform will strengthen the fundraising component and improve trust and transparency for donors. Additional security features such as stronger verification mechanisms and fraud detection can also be incorporated.

Future improvements will also include scaling the recommendation system by incorporating more advanced personalization techniques, as well as optimizing backend performance through caching, load balancing, and database scaling strategies. These enhancements will ensure that the system can support a larger user base and higher interaction volumes.

Overall, these developments will transform Kaarvan into a robust, scalable platform capable of supporting real-world social impact initiatives and fostering stronger community engagement.

8. Reflection

The development of Kaarvan was a valuable learning experience that combined both technical execution and real-world problem understanding. While the project successfully delivered its core functionality, the process highlighted several important insights regarding system design, team coordination, and dependency management.

One of the key challenges faced during development was adapting to new technologies, particularly React Native for mobile development and WebSocket-based real-time communication. These technologies introduced an initial learning curve, which impacted early development speed. However, overcoming these challenges significantly strengthened the team's ability to build scalable, full-stack applications and improved our understanding of modern application architectures.

Another major learning point was the importance of managing external dependencies. Delays from the industry partner affected aspects such as UI redesign which impacted deployment planning. This emphasized the need for clearer communication channels, and planning the UI before development starts when working with external stakeholders in real-world projects.

From a project management perspective, the team recognized the value of early planning and structured milestones. Allocating dedicated time for technology familiarization, testing, and integration earlier in the timeline would have reduced uncertainty and improved overall efficiency. Additionally, continuous testing and iterative development proved essential in identifying integration issues, particularly in real-time features and API communication.

Overall, the project not only achieved its functional objectives but also provided practical exposure to building a production-oriented system. It reinforced the importance of scalability, reliability, and user-centered design in software development. The experience has equipped the team with both technical skills and project management insights that will be valuable in future professional and academic work.

Appendix A. More Math

Appendix B. Mathematical Appendix

B.1 Recommendation System

Kaarvan uses Recombee, a third-party recommendation-as-a-service platform, to deliver personalized content to users. Recombee employs collaborative filtering and content-based filtering techniques internally to model user preferences and item similarity.

Collaborative filtering identifies users with similar interaction histories and recommends items those users engaged with. Given a set of users U and items I, the predicted preference score $\hat{r}_{ui}$ for user u and item i is estimated based on the weighted interactions of similar users:

$$\hat{r}_{ui} = \frac{\sum_{v \in N(u)} \text{sim}(u, v) \cdot r_{vi}}{\sum_{v \in N(u)} |\text{sim}(u, v)|} \tag{B.1}$$

where $N(u)$ is the set of users similar to u and $\text{sim}(u, v)$ is a similarity measure such as cosine similarity. The specific model parameters and training details are managed by Recombee and are not directly configured within the Kaarvan codebase.

Appendix C. Data

Appendix D. Data Appendix

D.1 Database Schema Overview

The Kaarvan database is managed using Prisma ORM with a PostgreSQL backend. The core entities are as follows:

- **User** — stores profile information, authentication provider, and notification tokens.
- **Community** — represents an NGO or social group, with ownership and membership relations.
- **Event** — linked to a community, tracks RSVP state per user.
- **Fundraiser** — linked to a community, stores target amount and description.
- **Discussion** — threaded posts within a community, with nested comments.
- **Message** — real-time chat messages scoped to a community room.

D.2 Key Relationships

A `User` can be a member of many `Community` entities and can RSVP to many `Event` entities. Each `Community` has one owner (`User`) and can have many `Events`, `Fundraisers`, `Discussions`, and `Messages`. `Discussion` entries support nested `Comment` records linked back to the authoring `User`.

Appendix E. Code

Appendix F. Code Appendix

F.1 Real-Time Messaging with Socket.io

The following snippet shows the server-side event handler for sending and broadcasting messages within a community chat room. Messages are persisted to the database before being emitted to other connected clients.

Listing F.1: Socket.io message handler

```
socket.on("sendMessage", async ({ roomId, senderId, content })
    => {
  try {
    const message = await prisma.message.create({
      data: { roomId, senderId, content },
      include: { sender: true },
    });
    io.to(roomId).emit("newMessage", message);
  } catch (error) {
    socket.emit("error", { message: "Failed to send message."
       });
  }
});
```

F.2 Google OAuth Integration

Authentication is handled through Google Sign-In using Expo's auth session library. The ID token returned by Google is verified server-side and used to create or retrieve the user record.

Listing F.2: Google Sign-In handler

```
const [request, response, promptAsync] = Google.useAuthRequest
   ({
  clientId: GOOGLE_CLIENT_ID,
  redirectUri: makeRedirectUri({ useProxy: true }),
});

useEffect(() => {
  if (response?.type === "success") {
    const { id_token } = response.params;
    verifyTokenWithBackend(id_token);
  }
}, [response]);
```

F.3 Recombee Recommendation Integration

User activity is logged to Recombee to build a preference model, and recommendations are fetched on the home screen to surface relevant communities and events.

Listing F.3: Fetching recommendations from Recombee

```
const getRecommendations = async (userId) => {
  const response = await recombee.send(
    new rqs.RecommendItemsToUser(userId, 10, {
      scenario: "homepage",
      returnProperties: true,
    })
  );
  return response.recomms;
};
```

References

[1] M. Wade. "The giving layer of the internet: A critical history of GoFundMe's reputation management, platform governance, and communication strategies in capturing peer-to-peer and charitable giving markets". In: *Journal of Philanthropy and Marketing* (2022). DOI: 10.1002/nvsm.1777.

[2] *GlobalGiving UK – Crowdfunding for grassroots organisations around the world.* GlobalGiving. 2025. URL: https://www.nesta.org.uk/blog/globalgiving-uk-crowdfunding-for-grassroots-organisations-around-the-world/.

[3] *VolunteerMatch is now part of Idealist! For Volunteering.* VolunteerMatch. 2025. URL: https://www.idealist.org/volunteermatch.

[4] *Best Volunteer Management Software | Golden.* Golden. 2025. URL: https://goldenvolunteer.com.

[5] *GivePulse | Giving Platform supporting your community and ...* GivePulse. 2023. URL: https://learn.givepulse.com.

[6] *POINT: All-in-one Volunteer Management Platform and App.* POINT. 2025. URL: https://pointapp.org.

[7] *Top 5 Online Donation Websites in Pakistan.* Transparent Hands. 2025. URL: https://www.transparenthands.org/list-of-top-5-online-donation-websites-in-pakistan/.

[8] *Give zakat or sadqah with easypaisa donations!* Easypaisa. 2025. URL: https://easypaisa.com.pk/donations/.

[9] *LaunchGood, the world's largest crowdfunding platform.* LaunchGood. 2025. URL: https://www.launchgood.com.

[10] X. Li. "An Improved Collaborative Filtering Recommendation Algorithm". In: *Journal of Applied Mathematics* (2019). DOI: 10.1155/2019/3560968.

[11] Y. Li and X. Lin. “Study of collaborative filtering recommendation with user clustering incorporating implicit social relationships and trust relationships”. In: *PLoS ONE* 20.10 (2025). DOI: 10.1371/journal.pone.0332998.

[12] LinkedIn Engineering. *Viral spam content detection at LinkedIn*. 2021. URL: https://www.linkedin.com/blog/engineering/trust-and-safety/viral-spam-content-detection-at-linkedin.

[13] M. Al-Ghobari, A. Muneer, and S. M. Fati. “Location-aware personalized traveler recommender system (LAPTA) using collaborative filtering KNN”. In: *Computers, Materials & Continua* 69.2 (2021), pp. 1553–1570. DOI: 10.32604/cmc.2021.016348.

[14] Vedashree and P. Sandhya. “Location-based personalized recommendation system”. In: *International Research Journal of Modernization in Engineering Technology and Science* 5.9 (2023), pp. 195–204.

[15] R. Jiang, R. Yang, and W. Lin. “Community-Aware Social Community Recommendation”. In: *arXiv preprint* (2025). URL: https://arxiv.org/abs/2508.05107v2.

[16] K. Kaur and K. S. Dhindsa. “TrendNet Algorithm: Towards Personalization of Twitter Trends”. In: *International Journal of Recent Technology and Engineering (IJRTE)* (2019).

[17] ConnectyCube Team. *WebSockets vs. Firebase: which is best for real-time chat?* 2025. URL: https://connectycube.com/2025/07/17/websockets-vs-firebase-which-is-best-for-real-time-chat/.

[18] Ably Real-Time. *XMPP vs WebSocket: Which is best for chat apps?* 2025. URL: https://ably.com/topic/xmpp-vs-websocket.